

Winter School of AI and Robotics [REPORT]

| Table of Contents

Day 1 - Session 1

Python Installation

Python Basics

Task: Write a program to run Fibonacci Series

Task: Write a program to find nth element in Fibonacci Sequence.

NumPy Installation

Task: Slice a 4×4 2D array and make it 3×3, containing elements of the bottom right 9 elements.

Task: Write a program to multiply two matrices of different dimensions.

OpenCV Library

Image Creation on CV2

Drawing a Line on the created image

Drawing an Ellipse on the Created Image

Drawing a Rectangle on the Created Image

Reading a Local Image using OpenCV

Day 1 - Session 2

Task: Write a code to show the projectile motion of a ball.

Task: Write a code of a moving box in 2D, which bounces when hits the boundary of the canvas.

Task: Write a code with a ball bouncing as in task 2, with a static circle on the canvas, such that the canvas closes when the ball touches the circle.

Task: Write a code for a simple Pacman program of the ball opening and closing its mouth with keyboard input

Day 2 - Session 1

Types of Images

Color Spaces

RGB Color Spaces

HSV Color Spaces

CMYK Color Spaces

MATPLOTLIB Module

Changing of Color Spaces (Using OpenCV)

Rotating an Image using OpenCV

Rotating an Image 90 degrees without using OpenCV

Rotating an Image 180 degrees without using OpenCV

Padding using OpenCV

Threshold Value in OpenCV

Task: Download a Greyscale image of a coin and find the threshold value above which you can see the circle around the coin

Task: Take any Image and convert it from RGB to Greyscale without using OpenCV

Task: Take any Image and convert it from RGB to Black and White without using OpenCV

Camera Videos using OpenCV

Standard Code

Task: Take any short Video and Resize it. Try drawing a circle over the region of interest or any other shape.

Day 2 - Session 2

Image Segmentation

Clustering

K-Means Clustering

Scatter Plot (MATPLOTLIB) -

Scatter Plot for Random points in a 2D Array -

Task: Write a program to execute K-means clustering of the above set of points.

Task: Write a program to execute K-means clustering for given two images, used RGB coordinates.

Day 3 - Session 2

Kernels

Gaussian Blur

Bilateral Filtering -

Edge Detection-

Methods of Edge Detection -

Sobel Method -

Task: Using Sobel Method, detect the edges of any image.

Laplace Method -

Task: Using Laplace Method, detect the edges of any image.

Gradients -

Canny Edge Detection -

Task: Using Canny Edge Detection, detect edges of any image

Hough Transform -

Day 4 - Session 1

Machine Learning for Computer Vision

Optimizing parameters -

Gradient Descent Method -

Task: Write the code for linear regression using gradient descent method on the California Housing Dataset.

Task: Write a program to implement Gradient Descent on a Simple Dataset and find the best fit line

Day 4 - Session 2

Logistic Regression

Task: Write a program to implement Gradient Descent on a Simple Dataset for Logistic Regression.

Universal Approximation Theorem

Convolutional Neural Networks

Parameters and Hyperparameters

Introduction of a new Loss function -

Kernels Continued

Training Models

Gradient Descent

Day 5 - Session 1

How to Represent Graphs

Edge List -

Adjacency List -

Adjacency Matrix -

Graph Traversal -

Graph to be used for BFS, DFS and Dijkstra's Algorithm Implementation

Edge List Representation of Above Graph

Breadth First Search -

Task: Write a general code to implement BFS algorithm

Task: Implement BFS Algorithm on the given graph.

Depth First Search -

Task: Write a general code to implement DFS algorithm

Task: Implement DFS Algorithm on the given graph.

Dijkstra's Algorithm

Task: Implement Dijkstra's Algorithm on the given graph.

Day 5 - Session 2

SLAM

Task: Write the general code for A* Algorithm (When diagonal movement is not allowed)

Task: Write the general code for A* Algorithm (When diagonal movement is allowed)

Task: Take an image, convert it to black and white and detect the edges using Canny Edge Detection. Pick two points and implement A* Algorithm to find the optimal path between the points. Consider the edges to be obstacles.

Day 6 - Session 1

Perform the following tasks to implement algorithms on this given maze.

Probabilistic Roadmap (PRM)

Task: Implement PRM on the given maze.

Rapidly-exploring Random Trees (RRT)

Task: Implement RRT on the given maze.

Rapidly-exploring Random Trees Star (RRT*)

Task: Implement RRT* on the given maze.

Final Day Project

Python Code

Day 1 - Session 1

Python Installation

- Everyone who had higher version of python (3.14) had to uninstall and reinstall lower version to stay on track.
- Google classroom was created and everyone was told to join. On this classroom, the assignments etc. will be posted.

Python Basics

- Variables, operators, loops etc

Task: Write a program to run Fibonacci Series

```
i = 1      # Initialize variable i with value 1
j = 1      # Initialize variable j with value 1

print(i)   # Print the initial value of i
print(j)   # Print the initial value of j

for k in range(1, 6): # Loop runs 5 times with k taking values from 1 to 5
    m = j          # Store the current value of j in temporary variable m
    j = j + i      # Update j to be the sum of j and i
    i = m          # Update i to the old value of j (stored in m)
    print(j)       # Print the updated value of j
```

- Simple Data structures were covered -
 1. Dictionaries
 2. Sets
 3. Strings (Arrays of Characters)
 4. Lists
 5. Tuples
- Functions (Syntax, meaning etc)

- Exception Handling (try, except, finally)

Task: Write a program to find nth element in Fibonacci Sequence.

```
j = int(input("Which element of the Fibonacci Sequence do you want?")) # Take user input for which Fibonacci element is required

list1 = [0 for p in range(j)] # Create a list of size j initialized with 0s

list1[0] = 1          # Set the first Fibonacci number to 1
list1[1] = 1          # Set the second Fibonacci number to 1

for i in range(2, j):    # Loop from the third element up to the j-th element
    list1[i] = list1[i-1] + list1[i-2] # Each Fibonacci number is the sum of the previous two

print(list1[j-1])        # Print the j-th Fibonacci number (0-based indexing)
```

NumPy Installation

pip install numpy

import numpy as np

- Many of the numpy functions were explained.

Task: Slice a 4x4 2D array and make it 3x3, containing elements of the bottom right 9 elements.

```
import numpy as np          # Import NumPy library for numerical operations

x = np.array([[0, 0, 0, 0],  # Create a 2D NumPy array (matrix)
              [0, 1, 2, 3],
              [0, 4, 5, 6],
              [0, 7, 8, 9]])

y = x[1:, 1:]              # Slice the array: remove first row and first column
print(y)                   # Print the resulting submatrix
```

Task: Write a program to multiply two matrices of different dimensions.

```
import numpy as np # Import NumPy for matrix and numerical operations

x = int(input("What is the number of rows in your first matrix? ")) # Number of rows in first matrix
y = int(input("What is the number of columns in your first matrix? ")) # Number of columns in first matrix
z = int(input("What is the number of columns in your second matrix? ")) # Number of columns in second matrix

m = np.full((x, y), 0, dtype=int) # Initialize first matrix with zeros (size x × y)
n = np.full((y, z), 0, dtype=int) # Initialize second matrix with zeros (size y × z)

for i in range(x): # Loop over rows of the first matrix
    for j in range(y): # Loop over columns of the first matrix
        m[i][j] = int(input("Enter element (Matrix 1): ")) # Take input for first matrix elements

for i in range(y): # Loop over rows of the second matrix
    for j in range(z): # Loop over columns of the second matrix
        n[i][j] = int(input("Enter element (Matrix 2): ")) # Take input for second matrix elements

final = np.full((x, z), 0, dtype=int) # Initialize the result matrix with zeros (size x × z)

def multiply(a, b): # Function to multiply two matrices a and b
    for i in range(x): # Loop over rows of the result matrix
        for j in range(z): # Loop over columns of the result matrix
            for k in range(y): # Loop for matrix multiplication logic
                final[i, j] = final[i, j] + a[i, k] * b[k, j] # Accumulate the product
    print(final) # Print the final multiplied matrix

multiply(m, n) # Call the function to multiply matrices m and n
```

OpenCV Library

pip install opencv-python

- Images are made up of pixels.
- Grayscale and RGB images.

Image Creation on CV2

- Syntax -

`np.full(shape, fill_value, dtype)`

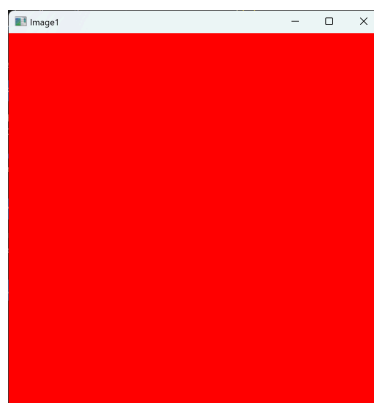
- Example-

`img = np.full((500, 500, 0), (255, 0, 0), dtype = np.uint8)`

```
import cv2                # Import OpenCV library for image processing
import numpy as np        # Import NumPy for numerical and array operations

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)
# Create a 500×500 image with 3 color channels (BGR),
# filled with red color (0, 0, 255) using unsigned 8-bit integers

cv2.imshow("Image1", img)    # Display the image in a window named "Image1"
cv2.waitKey(0)              # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()     # Close all OpenCV windows
```



Drawing a Line on the created image

- Syntax -

`cv2.line(img, pt1, pt2, color, thickness)`

- Example -

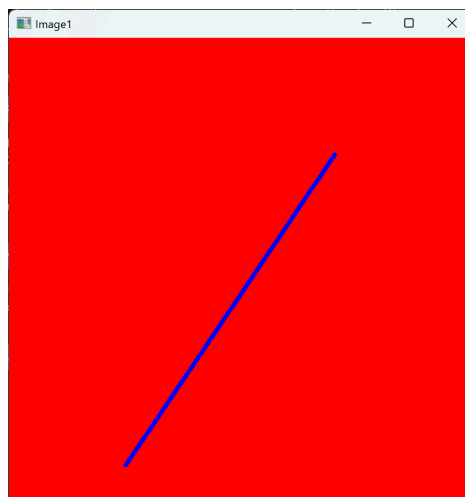
`cv2.line(img, (350, 125), (125, 460), (255, 0, 0), 3)`

```
import cv2                # Import OpenCV library for image processing
import numpy as np        # Import NumPy for numerical and array operations

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)
# Create a 500×500 image with 3 color channels (BGR),
# filled with red color (0, 0, 255)

cv2.line(img, (350, 125), (125, 460), (255, 0, 0), 3)
# Draw a blue line (BGR: 255, 0, 0) on the image
# Starting point: (350, 125), Ending point: (125, 460)
# Thickness of the line is 3 pixels

cv2.imshow("Image1", img)    # Display the image in a window named "Image1"
cv2.waitKey(0)              # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()     # Close all OpenCV windows
```



Drawing an Ellipse on the Created Image

- Syntax -

`cv2.ellipse(img, center, axes, angle, startAngle, endAngle, color, thickness)`

- Example -

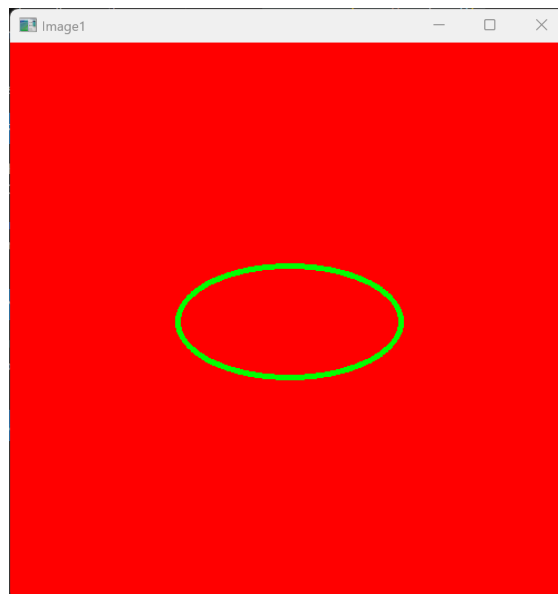
```
cv2.ellipse(img, (250, 250), (100, 50), 0, 0, 360, (0, 255, 0), 3)
```

```
import cv2                # Import OpenCV library for image processing
import numpy as np        # Import NumPy for numerical and array operations

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)
# Create a 500×500 image with 3 color channels (BGR),
# filled with red color (0, 0, 255)

cv2.ellipse(img, (250, 250), (100, 50), 0, 0, 360, (0, 255, 0), 3)
# Draw an ellipse on the image
# Center of ellipse: (250, 250)
# Lengths of the major and minor axes: (100, 50)
# Angle of rotation: 0 degrees
# Start angle: 0 degrees, End angle: 360 degrees (full ellipse)
# Color of ellipse: green (BGR: 0, 255, 0)
# Thickness of the ellipse boundary is 3 pixels

cv2.imshow("Image1", img)    # Display the image in a window named "Image1"
cv2.waitKey(0)              # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()     # Close all OpenCV windows
```



Drawing a Rectangle on the Created Image

- Syntax -

`cv2.rectangle(img, pt1, pt2, color, thickness)`

where pt1 is coordinate of top left corner

pt2 is coordinate of bottom right corner

- Example -

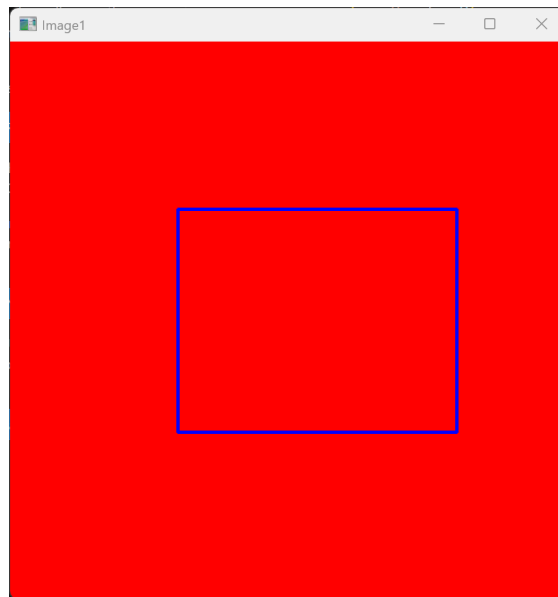
`cv2.rectangle(img, (150, 150), (400, 350), 255, 2)`

```
import cv2                # Import OpenCV library for image processing
import numpy as np        # Import NumPy for numerical and array operations

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)
# Create a 500×500 image with 3 color channels (BGR),
# filled with red color (0, 0, 255)

cv2.rectangle(img, (150, 150), (400, 350), 255, 2)
# Draw a rectangle on the image
# Top-left corner: (150, 150)
# Bottom-right corner: (400, 350)
# Color value: 255 (interpreted as white in grayscale intensity for drawing)
# Thickness of the rectangle border is 2 pixels

cv2.imshow("Image1", img)    # Display the image in a window named "Image1"
cv2.waitKey(0)              # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()     # Close all OpenCV windows
```



Reading a Local Image using OpenCV

```
import cv2                # Import OpenCV library for image processing
import numpy as np        # Import NumPy (not directly used here, but
                           # commonly used with OpenCV)

img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.
png")
# Read the image from the specified file path in default (BGR color) mode

img = cv2.resize(img, (1000, 500))
# Resize the color image to width = 1000 pixels and height = 500 pixels

img_gray = cv2.imread(
    r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png",
    cv2.IMREAD_GRAYSCALE
)
# Read the same image in grayscale mode

img_gray = cv2.resize(img_gray, (1000, 500))
# Resize the grayscale image to the same dimensions as the color image

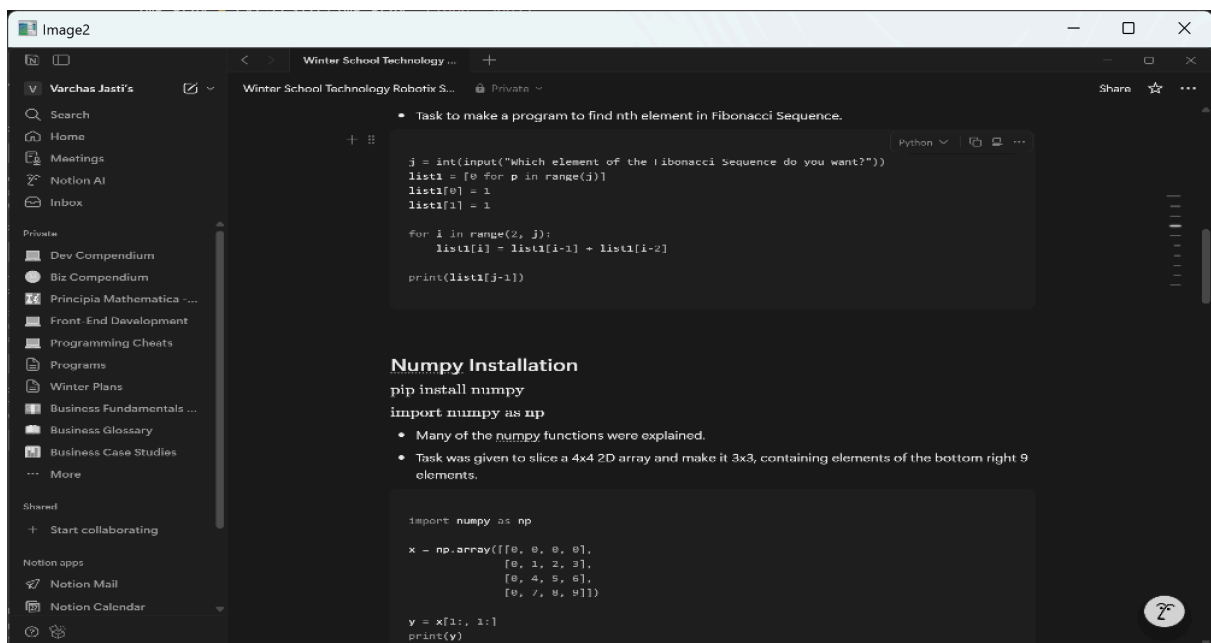
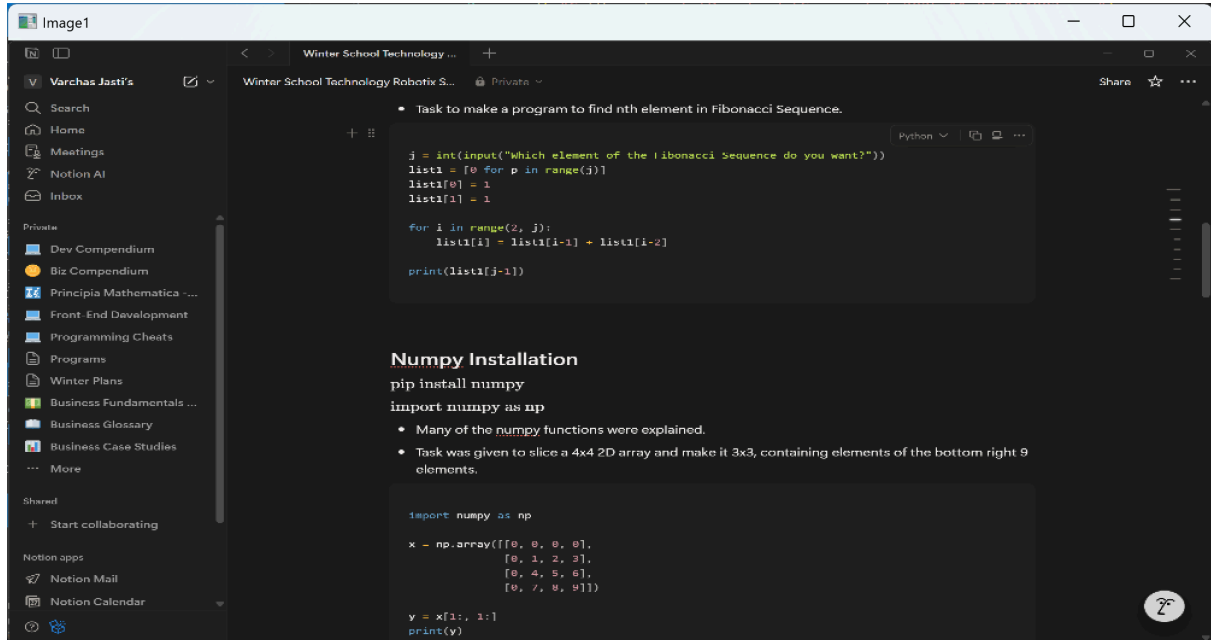
cv2.imshow("Image1", img)      # Display the color image in a window na
                               # med "Image1"
cv2.imshow("Image2", img_gray) # Display the grayscale image in a win
                               # dow named "Image2"
```

cv2.waitKey(0)

Wait indefinitely until a key is pressed

cv2.destroyAllWindows()

Close all OpenCV windows



Day 1 - Session 2

Task: Write a code to show the projectile motion of a ball.

```
import cv2                                # Import OpenCV for displaying graphics
import numpy as np                        # Import NumPy for array creation and manipulation

width, height = 500, 500                 # Define the width and height of the window

x, y = 0, 450                            # Initial position of the projectile (starting near bottom-left)
vx = 4                                   # Horizontal velocity
vy = -12                                 # Initial vertical velocity (negative for upward motion)
gravity = 0.4                             # Constant acceleration due to gravity

while True:                               # Infinite loop to animate the motion
    frame = np.full((height, width, 3), (0, 0, 255), dtype=np.uint8)
    # Create a red background frame of given dimensions

    x += vx                               # Update horizontal position using horizontal velocity
    vy += gravity                           # Increase vertical velocity due to gravity
    y += vy                                # Update vertical position using vertical velocity

    cv2.circle(frame, (int(x), int(y)), 10, (255, 255, 255), -1)
    # Draw the projectile as a filled white circle at the current position

    cv2.imshow("Simple Projectile", frame)
    # Display the current frame in a window titled "Simple Projectile"

    if x > width or y > height:            # Check if the projectile has gone out of bounds
        x, y = 0, 450                     # Reset projectile to initial position
        vy = -12                           # Reset vertical velocity

    if cv2.waitKey(20) & 0xFF == ord('q'):
        break                               # Exit the loop if the 'q' key is pressed
```

```
cv2.destroyAllWindows()
```

```
# Close all OpenCV windows
```

Task: Write a code of a moving box in 2D, which bounces when hits the boundary of the canvas.

```
import cv2                                # Import OpenCV for image creation and display
import numpy as np                        # Import NumPy for array operations
import keyboard as kb                    # Import keyboard module to detect key presses

i = 0                                    # Initialize a variable (not used further in the code)
x1 = 10                                  # Top-left x-coordinate of the rectangle
y1 = 10                                  # Top-left y-coordinate of the rectangle
x2 = 50                                  # Bottom-right x-coordinate of the rectangle
y2 = 50                                  # Bottom-right y-coordinate of the rectangle
velx = 10                                # Velocity of the rectangle along the x-axis
vely = 5                                 # Velocity of the rectangle along the y-axis

while True:                              # Infinite loop for animation
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)
    # Create a 500×500 black background image

    cv2.rectangle(img, (x1, y1), (x2, y2), 255, -1)
    # Draw a filled rectangle (color intensity 255) using the given coordinates

    cv2.imshow("Image1", img)             # Display the image in a window named "Image1"
    cv2.waitKey(50)                       # Wait for 50 milliseconds between frames

    x1 += velx                             # Update rectangle position along x-axis (left side)
    x2 += velx                             # Update rectangle position along x-axis (right side)
    y1 += vely                             # Update rectangle position along y-axis (top side)
    y2 += vely                             # Update rectangle position along y-axis (bottom side)
```

```

    if ((x1 + x2) / 2 > 470 or (x1 + x2) / 2 < 30):
        velx = -velx          # Reverse x-direction velocity when hitting vertical boundaries

    if ((y1 + y2) / 2 > 470 or (y1 + y2) / 2 < 30):
        vely = -vely          # Reverse y-direction velocity when hitting horizontal boundaries

    if (kb.is_pressed("ctrl")):
        break                  # Exit the loop when the Ctrl key is pressed

cv2.destroyAllWindows()      # Close all OpenCV windows

```

Task: Write a code with a ball bouncing as in task 2, with a static circle on the canvas, such that the canvas closes when the ball touches the circle.

```

import cv2                    # Import OpenCV for image processing and display
import numpy as np           # Import NumPy for array creation and manipulation
import math                   # Import math module for mathematical operations
import keyboard as kb         # Import keyboard module to detect key presses

velx = 10                     # Velocity along the x-axis
vely = 5                      # Velocity along the y-axis

x, y = 250, 250               # Initial position of the moving object
h = 150                       # Fixed reference point for distance calculation

while True:                   # Infinite loop for continuous animation
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)
    # Create a 500×500 black background image

    cv2.ellipse(img, (x, y), (30, 30), 0, 0, 360, (0, 0, 255), -1)

```

```

# Draw a filled red circle (ellipse) representing the moving object

cv2.ellipse(img, (h, h), (30, 30), 0, 0, 360, (255, 0, 0), 2)
# Draw a blue outlined circle (ellipse) at a fixed position

cv2.imshow("Image1", img)      # Display the current frame
cv2.waitKey(50)                # Wait for 50 milliseconds between frames

x += velx                      # Update x-coordinate of the moving object
y += vely                      # Update y-coordinate of the moving object

if (x > 470 or x < 30):
    velx = -velx                # Reverse x-direction when hitting vertical boundaries

if (y > 470 or y < 30):
    vely = -vely                # Reverse y-direction when hitting horizontal boundaries

if (math.sqrt(((x - h) * (x - h)) + ((y - h) * (y - h))) < 60):
    break                      # Stop the loop if the moving object is close to the fixed object

if (kb.is_pressed("ctrl")):
    break                      # Exit the loop when the Ctrl key is pressed

cv2.destroyAllWindows()        # Close all OpenCV windows

```

Task: Write a code for a simple Pacman program of the ball opening and closing its mouth with keyboard input

```

import cv2                      # Import OpenCV for image creation and display
import numpy as np              # Import NumPy for array operations
import keyboard as kb           # Import keyboard module to detect key presses

i = 0                          # Angle control variable for animation
j = 1                          # Direction control variable for angle increment/decr

```

```

ement

vel = 3                                # Velocity of movement in pixels

x, y = 25, 25                          # Initial position of the ellipse

while True:                            # Main infinite loop
    while (kb.is_pressed("right")):    # While the right arrow key is pressed
        img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)
        # Create a black background

        cv2.ellipse(img, (x, y), (15, 15), 0, i, 360 - i, (255, 0, 0), -1)
        # Draw a filled blue ellipse facing right with animated arc angles

        cv2.imshow("Image1", img)      # Display the image
        cv2.waitKey(5)                 # Short delay for smooth animation

        i += j                         # Update animation angle
        if (x < 482):
            x += vel                    # Move ellipse to the right

        if (i == 44 or i == -44 or i == 0):
            j = -j                      # Reverse angle animation direction

    while (kb.is_pressed("down")):    # While the down arrow key is pressed
        img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

        cv2.ellipse(img, (x, y), (15, 15), 90, i, 360 - i, (255, 0, 0), -1)
        # Draw a filled ellipse rotated downward

        cv2.imshow("Image1", img)
        cv2.waitKey(5)

        i += j
        if (y < 482):
            y += vel                    # Move ellipse downward

        if (i == 44 or i == -44 or i == 0):
            j = -j

```

```

while (kb.is_pressed("left")):    # While the left arrow key is pressed
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

    cv2.ellipse(img, (x, y), (15, 15), 180, i, 360 - i, (255, 0, 0), -1)
    # Draw a filled ellipse rotated to the left

    cv2.imshow("Image1", img)
    cv2.waitKey(5)

    i += j
    if (x > 18):
        x -= vel            # Move ellipse to the left

    if (i == 44 or i == -44 or i == 0):
        j = -j

while (kb.is_pressed("up")):    # While the up arrow key is pressed
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

    cv2.ellipse(img, (x, y), (15, 15), 270, i, 360 - i, (255, 0, 0), -1)
    # Draw a filled ellipse rotated upward

    cv2.imshow("Image1", img)
    cv2.waitKey(5)

    i += j
    if (y > 18):
        y -= vel            # Move ellipse upward

    if (i == 44 or i == -44 or i == 0):
        j = -j

if (kb.is_pressed("ctrl")):
    break                    # Exit the program if Ctrl key is pressed

img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)
# Draw idle frame when no arrow key is pressed

```

```

cv2.ellipse(img, (x, y), (15, 15), 0, i, 360 - i, (255, 0, 0), -1)
# Draw ellipse in the current position

cv2.imshow("Image1", img)
cv2.waitKey(5)

i += j                # Continue idle animation
if (i == 44 or i == -44 or i == 0):
    j = -j            # Reverse animation direction if limits reached

cv2.destroyAllWindows()    # Close all OpenCV windows

```

Day 2 - Session 1

Types of Images

- Binary Images (Only Black and White)
- Greyscale (Black and White with concept of Intensity)
- Color Images (3 bands of monochrome colors, stores 3 bytes of data)

Color Spaces

RGB Color Spaces

- Most common.
- Additive in Nature.

HSV Color Spaces

- Hue(0-179), Saturation(0-255) and Value(0-255).
- Cylindrical representation of RGB color points.
- Attempts to depict colors as perceived by human eye.

CMYK Color Spaces

- Subtractive model. Subtracts colors red, green, blue, from white light
- White - Red = Cyan(C)

White - Green = Magenta(M)

White - Blue = Yellow(Y)

- K is color of Key Plate → Black

MATPLOTLIB Module

`pip install matplotlib`

- Used to plot graphs, charts, scattered graphs etc.

`import matplotlib.pyplot as plt`

- `plt.imshow` uses RGB color space with coordinates R, G, B coordinates only, whereas `cv2` uses B, G, R coordinates.

Changing of Color Spaces (Using OpenCV)

- Syntax

`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

- Example

`img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

```
import cv2                # Import OpenCV for image loading and processing
import numpy as np        # Import NumPy (commonly used with image arrays)
import matplotlib.pyplot as plt  # Import Matplotlib for displaying images

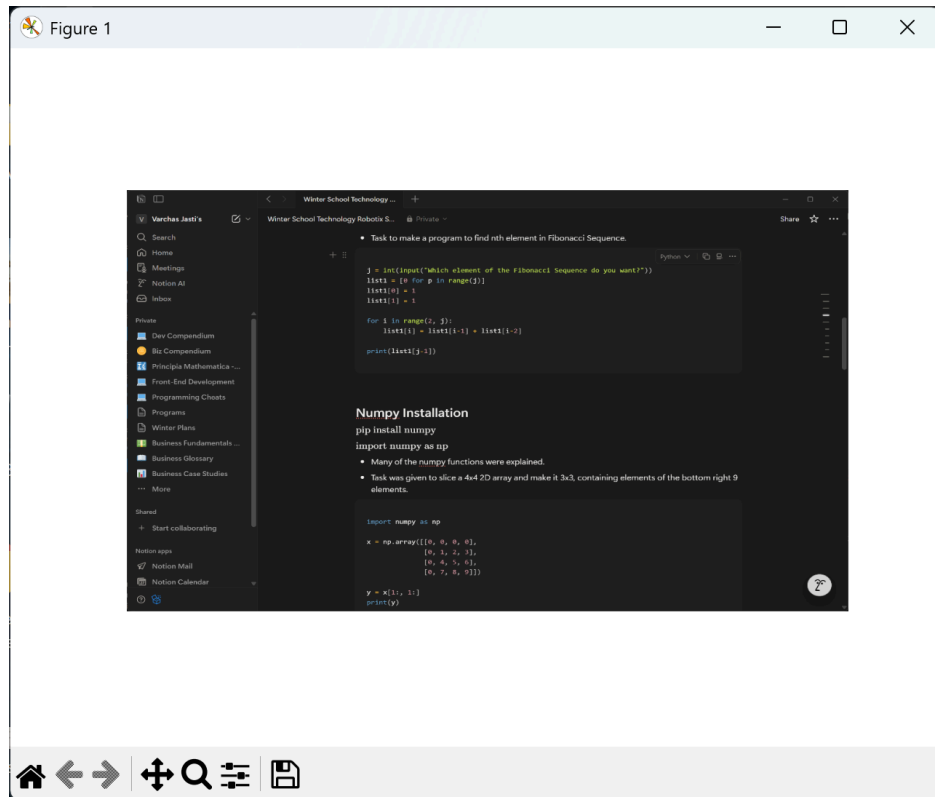
img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png")
# Read the image from the specified file path using OpenCV (default BGR color format)

img = cv2.resize(img, (1200, 700))
# Resize the image to width = 1200 pixels and height = 700 pixels

img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Convert the image from BGR (OpenCV format) to RGB (Matplotlib format)

plt.imshow(img_recolor)   # Display the RGB image using Matplotlib
```

```
plt.axis('off')          # Turn off axis markings
plt.show()              # Render the image window
```



Rotating an Image using OpenCV

```
import cv2                # Import OpenCV for image loading and processing
import numpy as np        # Import NumPy (commonly used with image arrays)
import matplotlib.pyplot as plt  # Import Matplotlib for displaying images

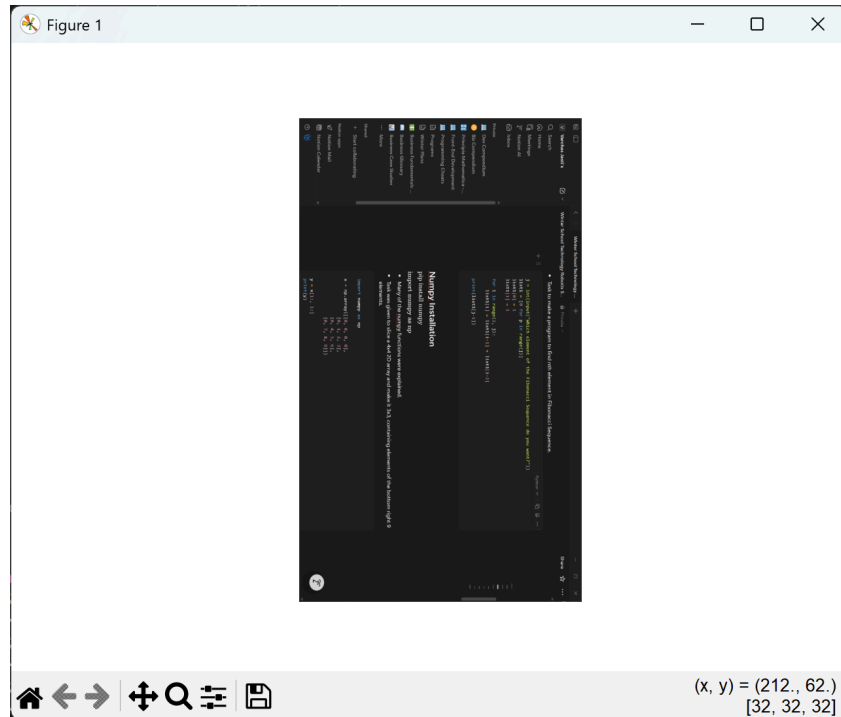
img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png")
# Read the image from the specified file path using OpenCV (default BGR color format)

img = cv2.resize(img, (1200, 700))
# Resize the image to width = 1200 pixels and height = 700 pixels

img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Convert the image from BGR (OpenCV format) to RGB (Matplotlib format)
```

```
img_rotated = cv2.rotate(img_recolor, cv2.ROTATE_90_CLOCKWISE)
# Rotate the RGB image by 90 degrees clockwise

plt.imshow(img_rotated)          # Display the rotated image using Matplotlib
plt.axis('off')                  # Turn off axis markings
plt.show()                       # Render the image window
```



Rotating an Image 90 degrees without using OpenCV

```
import cv2                      # Import OpenCV for image loading and display
import numpy as np              # Import NumPy for array creation and manipulation
import matplotlib.pyplot as plt # Import Matplotlib (not used directly here)

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path

img = cv2.resize(img, (732, 586))
# Resize the image to width = 732 and height = 586
```

```

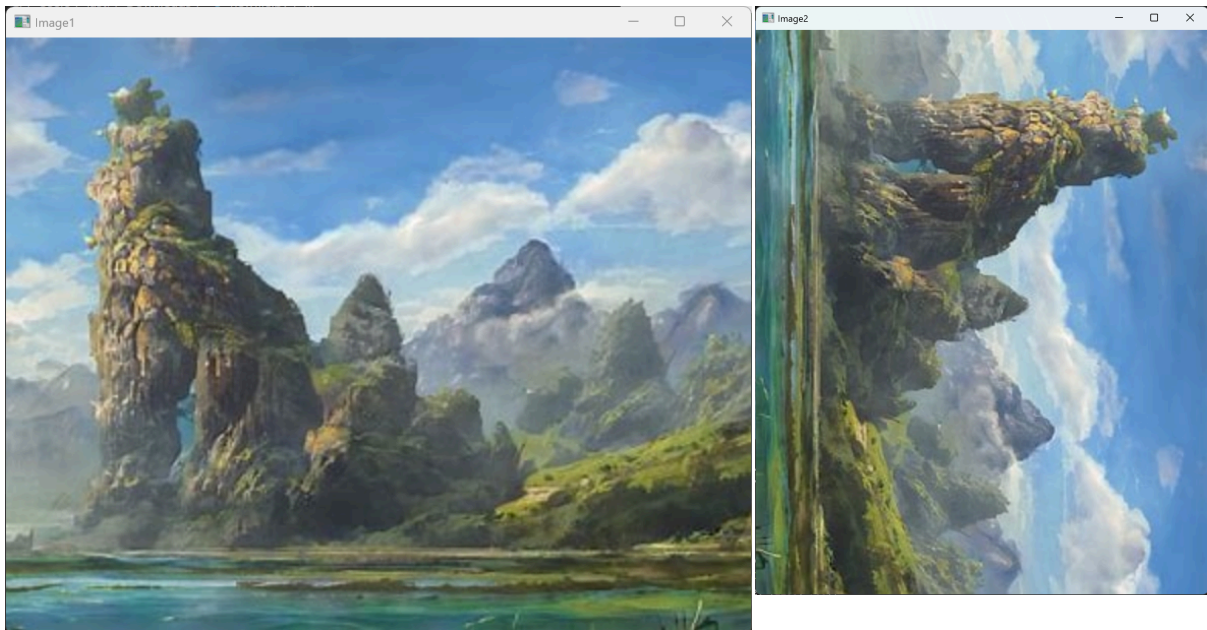
img_new = np.full((732, 586, 3), (0, 0, 0), dtype=np.uint8)
# Create a new black image array with swapped dimensions for manual rotation

for i in range(732):          # Loop over width of the resized image
    for j in range(586):      # Loop over height of the resized image
        img_new[i][j] = img[585 - j][i]
        # Assign pixels to rotate the image manually (90-degree rotation)

cv2.imshow("Image1", img)      # Display the original image
cv2.imshow("Image2", img_new)  # Display the newly created rotated image

cv2.waitKey(0)                # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()       # Close all OpenCV windows

```



Rotating an Image 180 degrees without using OpenCV

```

import cv2          # Import OpenCV for image loading and display
import numpy as np  # Import NumPy for array creation and manipulation
import matplotlib.pyplot as plt  # Import Matplotlib (not used directly here)

```

```

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path

img = cv2.resize(img, (732, 586))
# Resize the image to width = 732 and height = 586

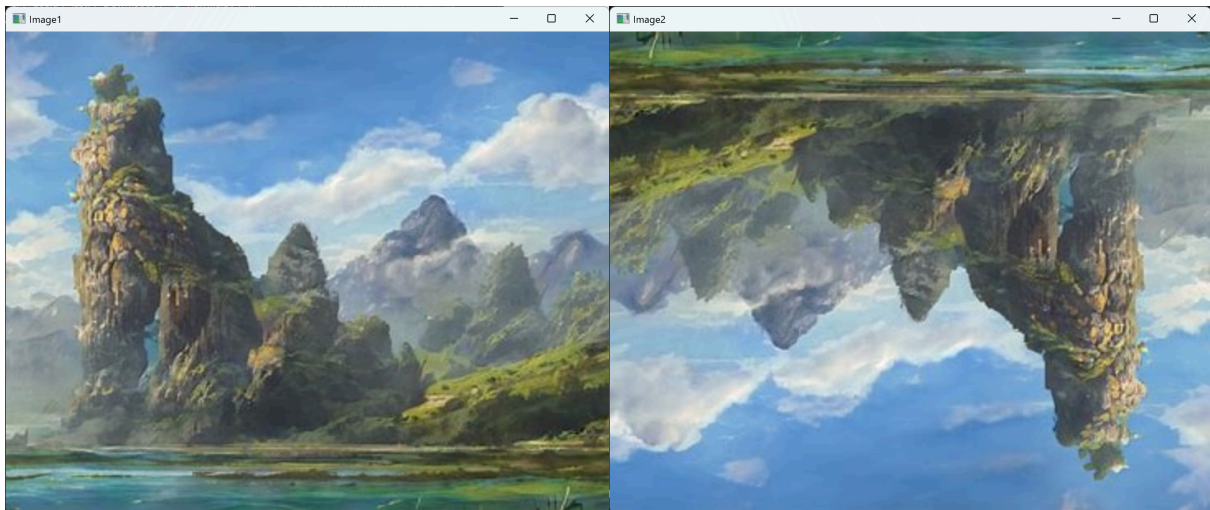
img_new = np.full((586, 732, 3), (0, 0, 0), dtype=np.uint8)
# Create a new black image array with dimensions swapped (for rotation)

for i in range(732):          # Loop over width of the original image
    for j in range(586):      # Loop over height of the original image
        img_new[j][i] = img[585 - j][731 - i]
        # Assign pixels from the original image to the new image
        # This mapping performs a manual 180-degree rotation

cv2.imshow("Image1", img)      # Display the original image
cv2.imshow("Image2", img_new)  # Display the transformed image

cv2.waitKey(0)                # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()       # Close all OpenCV windows

```



Padding using OpenCV

```

import cv2                                # Import OpenCV for image loading and processing
import numpy as np                        # Import NumPy (commonly used with image arrays)
import matplotlib.pyplot as plt          # Import Matplotlib for displaying images

img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png")
# Read the image from the specified file path using OpenCV (default BGR color format)

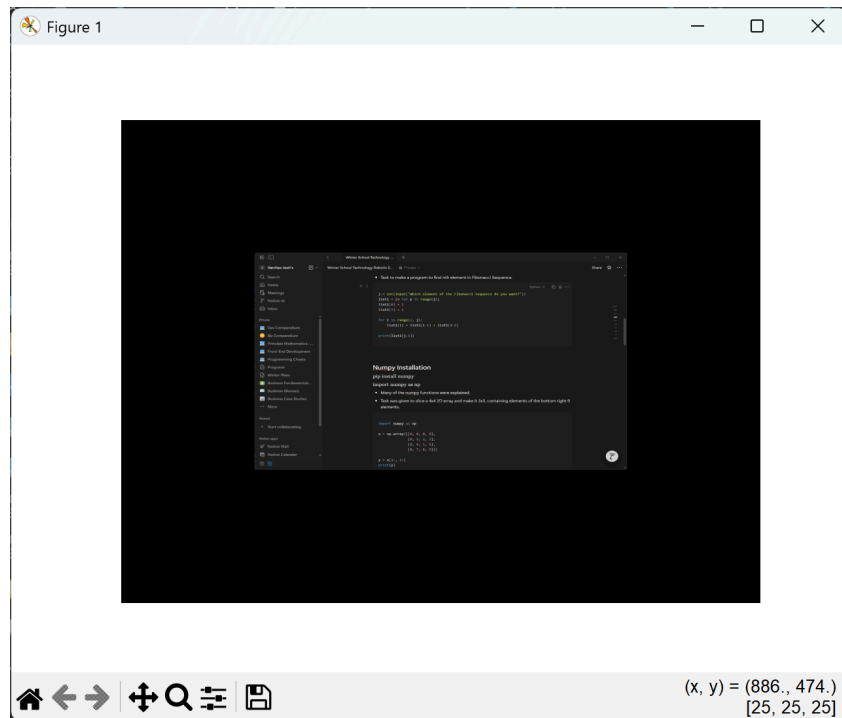
img = cv2.resize(img, (1200, 700))
# Resize the image to width = 1200 pixels and height = 700 pixels

img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Convert the image from BGR (OpenCV format) to RGB (Matplotlib format)

img_padded = cv2.copyMakeBorder(
    img_recolor,
    430, 430, 430, 430,
    cv2.BORDER_CONSTANT
)
# Add constant padding of 430 pixels on all sides of the image

plt.imshow(img_padded)                    # Display the padded image using Matplotlib
plt.axis('off')                           # Turn off axis markings
plt.show()                                # Render the image window

```



Threshold Value in OpenCV

Task: Download a Greyscale image of a coin and find the threshold value above which you can see the circle around the coin

```
import cv2                                # Import OpenCV for image loading and processing
import numpy as np                        # Import NumPy (commonly used with image arrays)
import matplotlib.pyplot as plt          # Import Matplotlib for displaying images

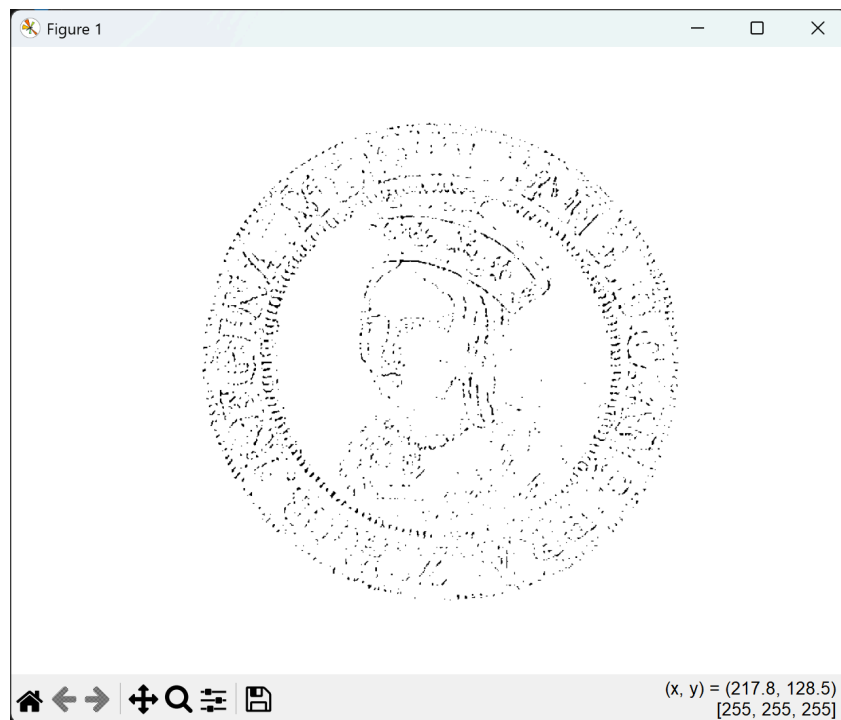
img = cv2.imread(r"C:\Users\jasti\Downloads\coin.jpg")
# Read the image from the specified file path using OpenCV

img = cv2.resize(img, (610, 616))
# Resize the image to width = 610 pixels and height = 616 pixels

ret, binary = cv2.threshold(img, 50, 255, cv2.THRESH_BINARY)
# Apply binary thresholding:
# Pixel values greater than 50 are set to 255 (white),
# and values less than or equal to 50 are set to 0 (black)
```

```
plt.imshow(binary, cmap='gray')    # Display the thresholded image in gray
scale
plt.axis('off')                    # Turn off axis markings
plt.show()                         # Render the image window
```

- Result: Threshold value is 50



Task: Take any Image and convert it from RGB to Greyscale without using OpenCV

```
import cv2                                # Import OpenCV for image loading and processing
import numpy as np                        # Import NumPy for array creation and manipulation
import matplotlib.pyplot as plt          # Import Matplotlib for displaying images

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path (BGR format)

img = cv2.resize(img, (732, 586))
# Resize the image to width = 732 pixels and height = 586 pixels

img_new = np.full((586, 732), 0, dtype=np.uint8)
```

```
# Create a new single-channel (grayscale) image initialized with zeros

for i in range(586):          # Loop over rows (height) of the image
    for j in range(732):      # Loop over columns (width) of the image
        img_new[i][j] = (img[i][j][0] + img[i][j][1] + img[i][j][2]) / 3
        # Convert each pixel to grayscale by averaging its B, G, and R values

plt.imshow(img_new, cmap='gray') # Display the grayscale image using Matplotlib
plt.axis('off')                 # Turn off axis markings
plt.show()                      # Render the image window
```





Task: Take any Image and convert it from RGB to Black and White without using OpenCV

```
import cv2                                # Import OpenCV for image loading and processing
import numpy as np                        # Import NumPy for array creation and manipulation
import matplotlib.pyplot as plt          # Import Matplotlib for displaying images

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path (BGR format)

img = cv2.resize(img, (732, 586))
# Resize the image to width = 732 pixels and height = 586 pixels

img_new = np.full((586, 732), 0, dtype=np.uint8)
# Create a new single-channel image initialized with zeros

for i in range(586):                      # Loop over rows (height) of the image
    for j in range(732):                  # Loop over columns (width) of the image
```

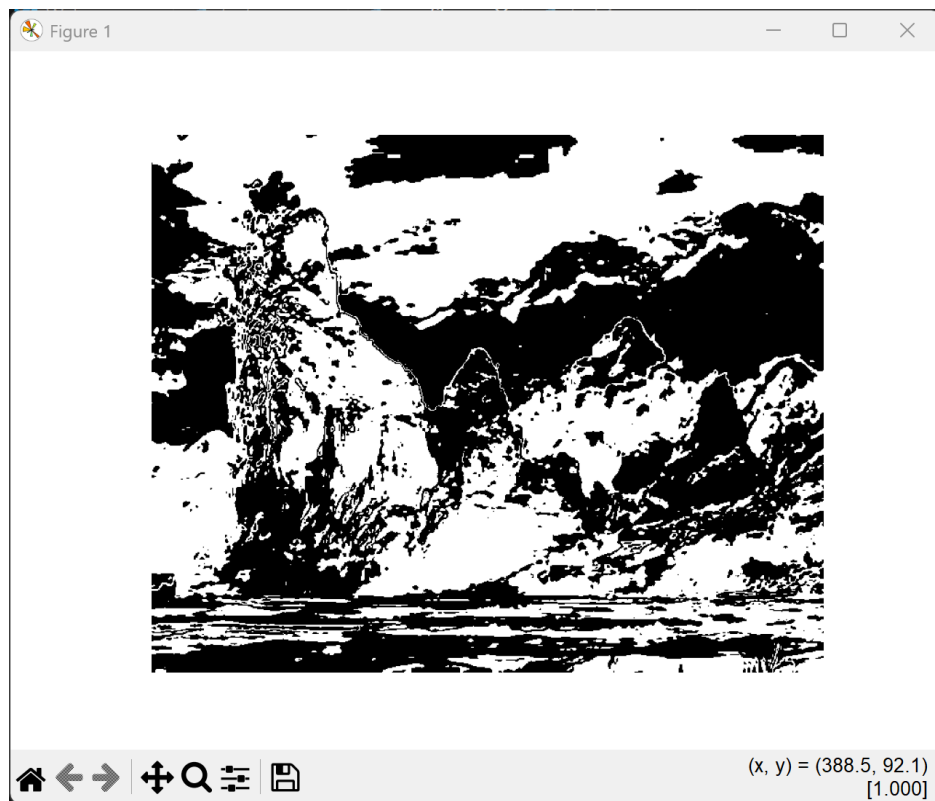
```

img_new[i][j] = (img[i][j][0] + img[i][j][1] + img[i][j][2]) / 3
# Convert each pixel to grayscale by averaging its B, G, and R values

if (img_new[i][j] < 50):
    img_new[i][j] = 0      # Set pixel to black if below threshold
else:
    img_new[i][j] = 1      # Set pixel to white (binary) if above threshold

plt.imshow(img_new, cmap='gray')    # Display the binary image using Matplotlib
plt.axis('off')                    # Turn off axis markings
plt.show()                         # Render the image window

```



Camera Videos using OpenCV

Standard Code

```

import numpy as np
import cv2 as cv
import keyboard as kb

```

```

cap = cv.VideoCapture(0)
if not cap.isOpened():
    print("Cannot Open Camera")
    exit()
while True:
    ret, frame = cap.read()
    if not ret:
        print("Can't receive frame (stream end?), Exiting ...")
        break

    cv.imshow('frame', frame)
    if cv.waitKey(1) == ord('q'):
        break
    if(kb.is_pressed('ctrl')):
        break

cap.release()
cv.destroyAllWindows()

```

Task: Take any short Video and Resize it. Try drawing a circle over the region of interest or any other shape.

```

import numpy as np                # Import NumPy (commonly used with image/video data)
import cv2 as cv                  # Import OpenCV (aliased as cv) for video processing
import keyboard as kb             # Import keyboard module to detect key presses

cap = cv.VideoCapture(r"C:\Users\jasti\Downloads\videoplayback.mp4")
# Create a VideoCapture object to read the video file from the given path

if not cap.isOpened():
    print("Cannot Open Camera")    # Print error message if video cannot be opened
    exit()                         # Exit the program

while True:                       # Loop to read and display video frames
    ret, frame = cap.read()        # Read one frame from the video

```

```

if not ret:
    print("Can't receive frame (stream end?), Exiting ...")
    break                # Exit loop if no frame is received (end of video)

cv.ellipse(frame, (240, 180), (100, 100), 0, 0, 360, (0, 0, 0), 8)
# Draw a black ellipse on the current video frame
# Center: (240, 180), Axes: (100, 100), Full ellipse, Thickness: 8

cv.imshow('frame', frame)      # Display the current frame in a window named 'frame'

if cv.waitKey(1) == ord('q'):
    break                    # Exit loop if 'q' key is pressed in the OpenCV window

if (kb.is_pressed('ctrl')):
    break                    # Exit loop if Ctrl key is pressed

cap.release()                 # Release the video capture object
cv.destroyAllWindows()        # Close all OpenCV windows

```

Day 2 - Session 2

Image Segmentation

- Splitting the image into different parts/segments based on characteristics of pixels.

Clustering

- Unsupervised learning to group data points based on similarity.
- Clustering in images are based on pixel similarity. Plot of each of these pixels in an RGB space.

K-Means Clustering

- Pick K random points (centroids).
- Assign each point to a centroid.

- Take the mean of coordinates of each cluster to be new centroid, discard the old centroid.
- Therefore centroids keep changing. If between two iterations centroid value does not change or changes very little, then it is said that the centroid is STABILISED.
- Distortion cost function-

$$J = \sum_{i=1}^m (x_i - \mu_{ci})^2$$
- Elbow Method to choose K -
 1. Start with K = 1.
 2. Find Distortion Cost function.
 3. Plot on graph.
 4. Continue for k = 2, 3, 4...
- The value of K for which J is minimum is the optimum value of K.
- Drawbacks of K means-
 1. Poor Initialization
 Solution for this is to choose k (fixed) random centroids multiple times (around 10) and choose the set of centroids among the 10 trials such that J is minimum.
 2. High Computation Cost

Scatter Plot (MATPLOTLIB) -

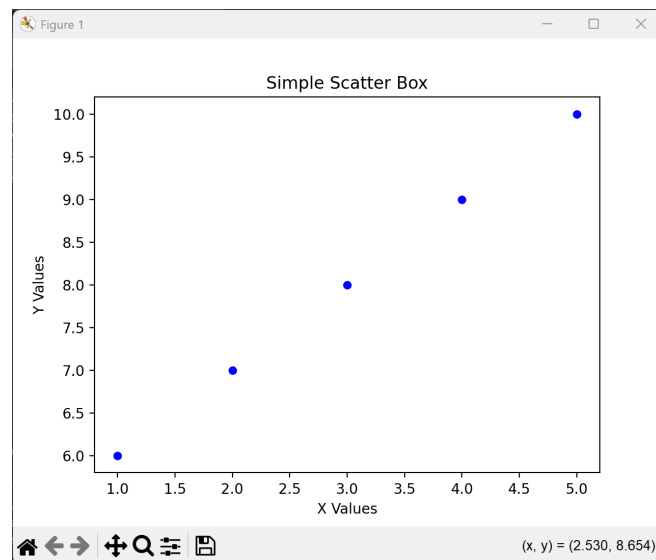
```
import cv2                # Import OpenCV (not used directly here)
import numpy as np        # Import NumPy (not used directly here)
import matplotlib.pyplot as plt    # Import Matplotlib for plotting

x = [1, 2, 3, 4, 5]       # Define x-axis data points
y = [6, 7, 8, 9, 10]     # Define y-axis data points

plt.scatter(x, y, s=100, marker='.', c='blue')
# Create a scatter plot:
# s=100 sets the size of each marker
# marker='.' specifies dot-shaped markers
# c='blue' sets the color of the points
```

```
plt.xlabel("X Values")          # Label the x-axis
plt.ylabel("Y Values")          # Label the y-axis
plt.title("Simple Scatter Box")  # Set the title of the plot

plt.show()                      # Display the scatter plot
```



Scatter Plot for Random points in a 2D Array -

```
import cv2                      # Import OpenCV (not used directly here)
import numpy as np              # Import NumPy for array manipulation
import matplotlib.pyplot as plt # Import Matplotlib for plotting

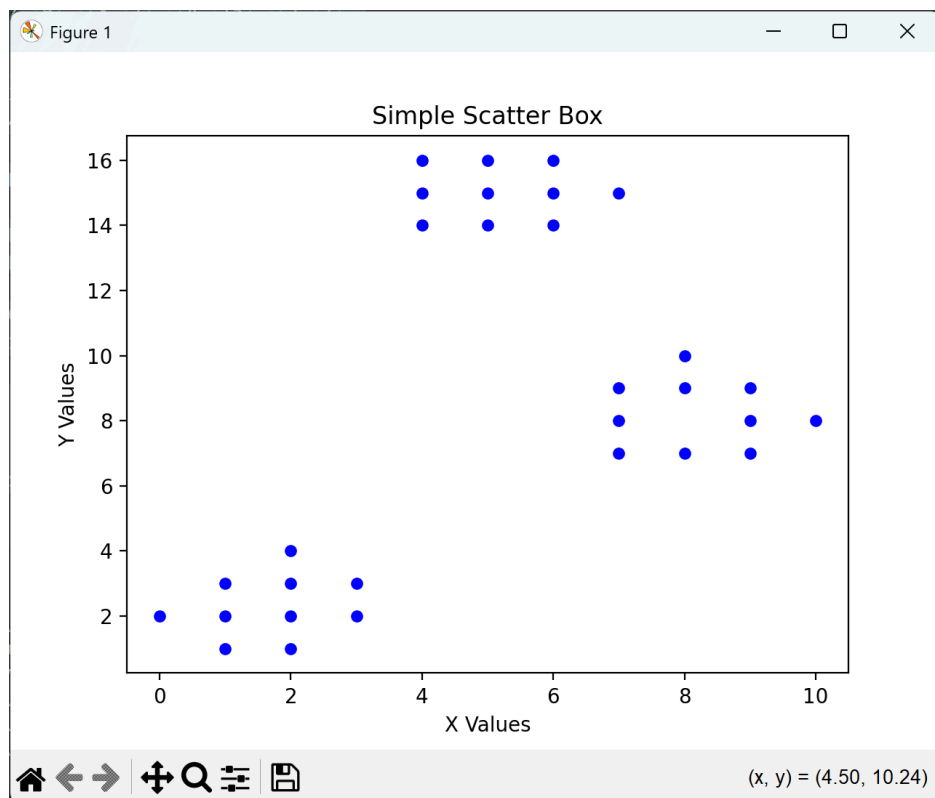
points = np.array([
    [9, 7], [2, 3], [6, 15], [8, 7], [1, 3],
    [5, 14], [7, 9], [3, 2], [4, 15], [2, 2],
    [8, 9], [1, 1], [6, 14], [5, 15], [7, 7],
    [9, 8], [4, 14], [2, 4], [0, 2], [8, 10],
    [5, 16], [3, 3], [10, 8], [7, 8], [1, 2],
    [6, 16], [4, 16], [2, 1], [9, 9], [7, 15]
])
# Create a NumPy array containing (x, y) coordinate pairs

x = points[:, 0]                # Extract all x-coordinates (first column)
y = points[:, 1]                # Extract all y-coordinates (second column)
```

```
plt.scatter(x, y, s=100, marker='.', c='blue')
# Plot the points using a scatter plot

plt.xlabel("X Values")          # Label the x-axis
plt.ylabel("Y Values")          # Label the y-axis
plt.title("Simple Scatter Box")  # Set the title of the plot

plt.show()                      # Display the scatter plot
```



Task: Write a program to execute K-means clustering of the above set of points.

```
import numpy as np          # Import NumPy for numerical and array operations
import matplotlib.pyplot as plt  # Import Matplotlib for plotting

points = np.array([
    [9, 7], [2, 3], [6, 15], [8, 7], [1, 3],
    [5, 14], [7, 9], [3, 2], [4, 15], [2, 2],
    [8, 9], [1, 1], [6, 14], [5, 15], [7, 7],
    [9, 8], [4, 14], [2, 4], [0, 2], [8, 10],
```

```

    [5, 16], [3, 3], [10, 8], [7, 8], [1, 2],
    [6, 16], [4, 16], [2, 1], [9, 9], [7, 15]
])
# Dataset containing 2D points for clustering

k = 3
# Number of clusters

centroids = points[np.random.choice(points.shape[0], k, replace=False)]
# Randomly initialize k centroids from the given points

for epoch in range(10):          # Run the K-means algorithm for a maximum
    of 10 iterations
        distances = np.linalg.norm(points[:, np.newaxis] - centroids, axis=2)
        # Compute the Euclidean distance from each point to each centroid

        labels = np.argmin(distances, axis=1)
        # Assign each point to the closest centroid (cluster index)

        new_centroids = np.array([
            points[labels == i].mean(axis=0) for i in range(k)
        ])
        # Compute new centroids by taking the mean of points in each cluster

        if np.allclose(centroids, new_centroids):
            break                # Stop if centroids do not change significantly

        centroids = new_centroids    # Update centroids for the next iteration

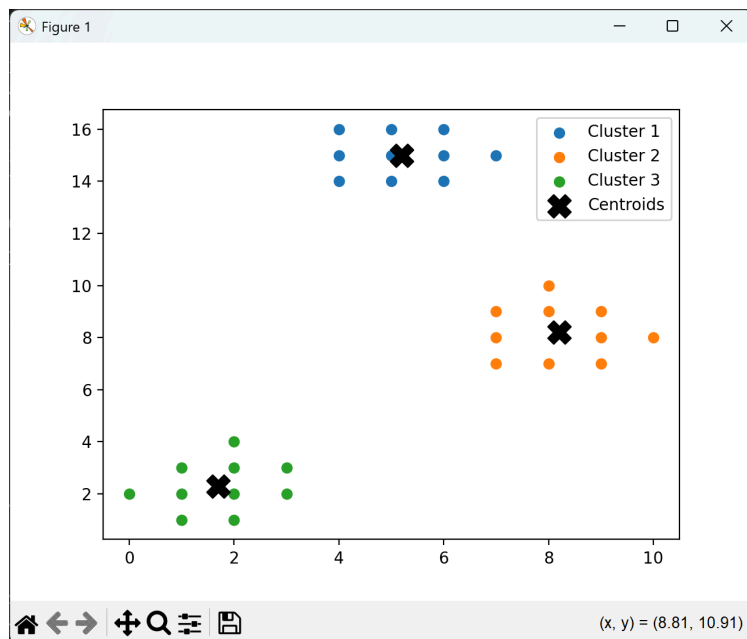
print("Final centroids:\n", centroids) # Print the final centroid coordinates
print("Cluster assignments:", labels) # Print cluster index for each point

for i in range(k):                # Plot each cluster separately
    cluster_points = points[labels == i]
    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        label=f'Cluster {i+1}'
    )

```

```
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='X',
    s=200,
    c='black',
    label='Centroids'
)
# Plot centroids as large black X markers

plt.legend()          # Show legend for clusters and centroids
plt.show()            # Display the final clustering plot
```



Task: Write a program to execute K-means clustering for given two images, used RGB coordinates.

```
import numpy as np          # Import NumPy for numerical and array operations
import cv2                  # Import OpenCV for image processing and display

image = cv2.resize(
    cv2.imread(r"C:\Users\jasti\Downloads\baboon_kmeans.jpeg"),
```

```

    (500, 500)
)
# Read the image from the given path and resize it to 500×500 pixels

clustered = np.zeros_like(image)
# Create an empty image array to store the clustered (output) image

k = 7
# Number of clusters for K-means

centroids = [
    image[np.random.randint(0, 500), np.random.randint(0, 500)]
    for _ in range(k)
]
# Randomly initialize k centroids by selecting random pixels from the image

print("Initial centroids:", centroids) # Print the initial centroid RGB values

for iteration in range(3):          # Run the K-means algorithm for a maximum
    of 3 iterations
        clusters = [[] for _ in range(k)]
        # Create empty lists to store pixel indices for each cluster

        print("Iteration", iteration)    # Print current iteration number

        for i in range(500):            # Loop over image rows
            for j in range(500):        # Loop over image columns
                dists = [
                    np.linalg.norm(c - image[i, j]) for c in centroids
                ]
                # Compute distance between the current pixel and each centroid (RGB
                space)

                kmin = np.argmin(dists)
                # Find the index of the closest centroid

                clusters[kmin].append((i, j))
                # Assign the pixel to the closest cluster

```

```

old_centroids = centroids.copy()
# Store old centroids to check for convergence

for i in range(k):          # Update each centroid
    if len(clusters[i]) > 0:
        rgb_cluster = np.array(
            [image[x, y] for x, y in clusters[i]]
        )
        # Collect RGB values of all pixels in the current cluster

        centroids[i] = np.mean(rgb_cluster, axis=0).astype(int)
        # Update centroid as the mean RGB value of the cluster

        print("Centroid", i, ":", centroids[i])
        # Print updated centroid value

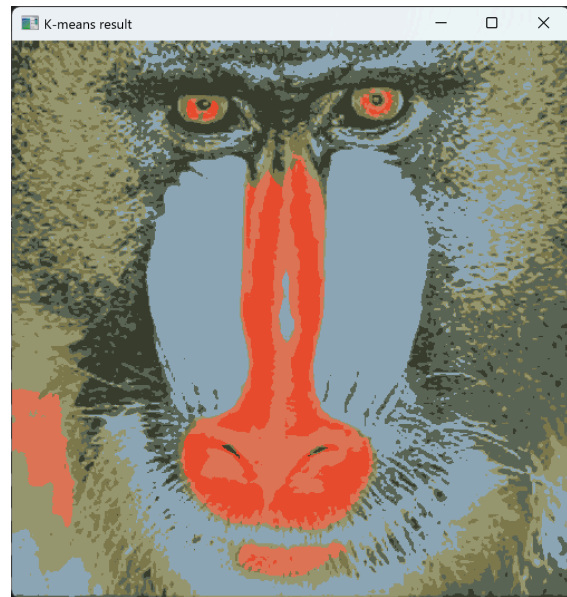
difference = np.abs(
    np.array(old_centroids) - np.array(centroids)
)
# Compute absolute difference between old and new centroids

if (np.sum(difference) < 10):
    break                # Stop if centroid change is very small (convergence)

for i in range(len(clusters)):    # Assign final cluster colors to the output image
    for x, y in clusters[i]:
        clustered[x, y] = centroids[i]
        # Set each pixel in the cluster to its centroid color

cv2.imshow("K-means result", clustered) # Display the clustered image
cv2.imshow("Actual Image", image)      # Display the original image
cv2.waitKey(0)                         # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()                # Close all OpenCV windows

```



```
import numpy as np                # Import NumPy for numerical and array operations
import cv2                       # Import OpenCV for image processing and display

image = cv2.resize(
    cv2.imread(r"C:\Users\jasti\Downloads\beach_kmeans.png"),
    (500, 500)
)
# Read the image from the given path and resize it to 500×500 pixels

clustered = np.zeros_like(image)
# Create an empty image array to store the clustered (output) image

k = 10
# Number of clusters for K-means

centroids = [
    image[np.random.randint(0, 500), np.random.randint(0, 500)]
    for _ in range(k)
]
# Randomly initialize k centroids by selecting random pixels from the image

print("Initial centroids:", centroids) # Print the initial centroid RGB values
```

```

for iteration in range(3):          # Run the K-means algorithm for a maximum
of 3 iterations
    clusters = [[] for _ in range(k)]
    # Create empty lists to store pixel indices for each cluster

    print("Iteration", iteration)   # Print current iteration number

    for i in range(500):            # Loop over image rows
        for j in range(500):        # Loop over image columns
            dists = [
                np.linalg.norm(c - image[i, j]) for c in centroids
            ]
            # Compute distance between the current pixel and each centroid (RGB
space)

            kmin = np.argmin(dists)
            # Find the index of the closest centroid

            clusters[kmin].append((i, j))
            # Assign the pixel to the closest cluster

        old_centroids = centroids.copy()
        # Store old centroids to check for convergence

    for i in range(k):              # Update each centroid
        if len(clusters[i]) > 0:
            rgb_cluster = np.array(
                [image[x, y] for x, y in clusters[i]]
            )
            # Collect RGB values of all pixels in the current cluster

            centroids[i] = np.mean(rgb_cluster, axis=0).astype(int)
            # Update centroid as the mean RGB value of the cluster

            print("Centroid", i, ":", centroids[i])
            # Print updated centroid value

        difference = np.abs(

```

```

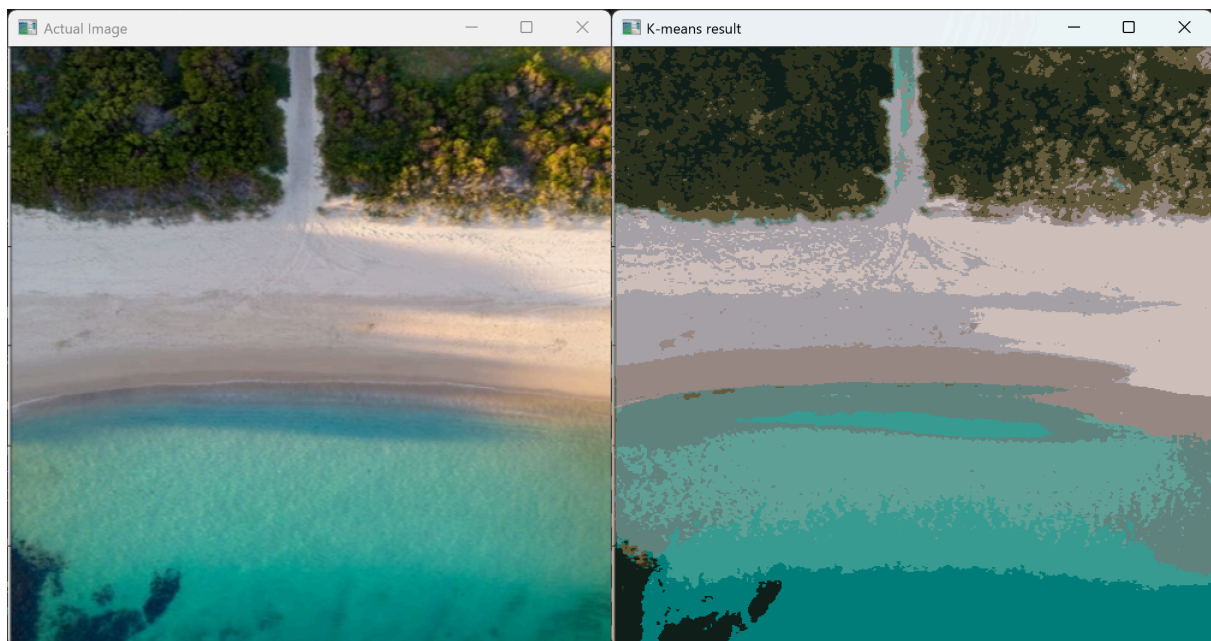
    np.array(old_centroids) - np.array(centroids)
)
# Compute absolute difference between old and new centroids

if (np.sum(difference) < 10):
    break                # Stop if centroid change is very small (convergence)

for i in range(len(clusters)):    # Assign final cluster colors to the output image
    for x, y in clusters[i]:
        clustered[x, y] = centroids[i]
        # Set each pixel in the cluster to its centroid color

cv2.imshow("K-means result", clustered) # Display the clustered image
cv2.imshow("Actual Image", image)      # Display the original image
cv2.waitKey(0)                         # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()                # Close all OpenCV windows

```



Day 3 - Session 2

Kernels

- Small matrices that are used to find out information about pixels.
- Mixes neighboring pixels to form a new value.

Gaussian Blur

- Blurring isn't always losing information.
- Blurring is important for cancelling noises in the image.
- The value of the final pixel is much closer to the weighted average of the neighboring pixels.
- Steps for Gaussian Blur -
 1. Create a blank canvas of the same size as image one.
 - 2.
- Implementation - Parameters
 1. Source Image
 2. Kernel size
 3. Standard deviation of the bell curve to be followed
- Formula to be followed -

$$K_{size} = 6\sigma + 1$$

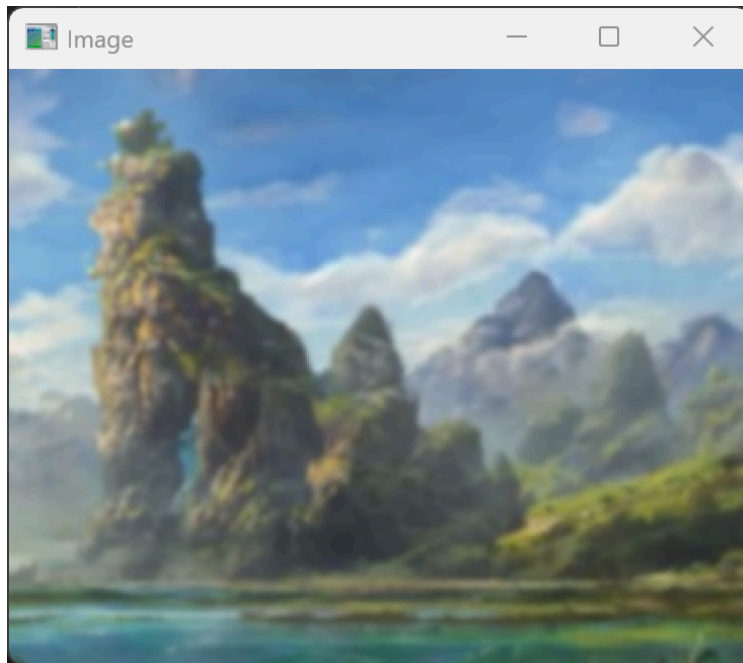
```
import cv2 as cv                # Import OpenCV (aliased as cv) for image processing
import numpy as np              # Import NumPy (commonly used with image arrays)

img = cv.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path

blur = cv.GaussianBlur(img, (9, 9), 1.5)
# Apply Gaussian blur to the image
# (9, 9) is the kernel size, and 1.5 is the standard deviation (sigma)

cv.imshow("Image", blur)        # Display the blurred image in a window named "Image"
```

```
cv.waitKey(0)                # Wait indefinitely until a key is pressed
cv.destroyAllWindows()       # Close all OpenCV windows
```



Bilateral Filtering -

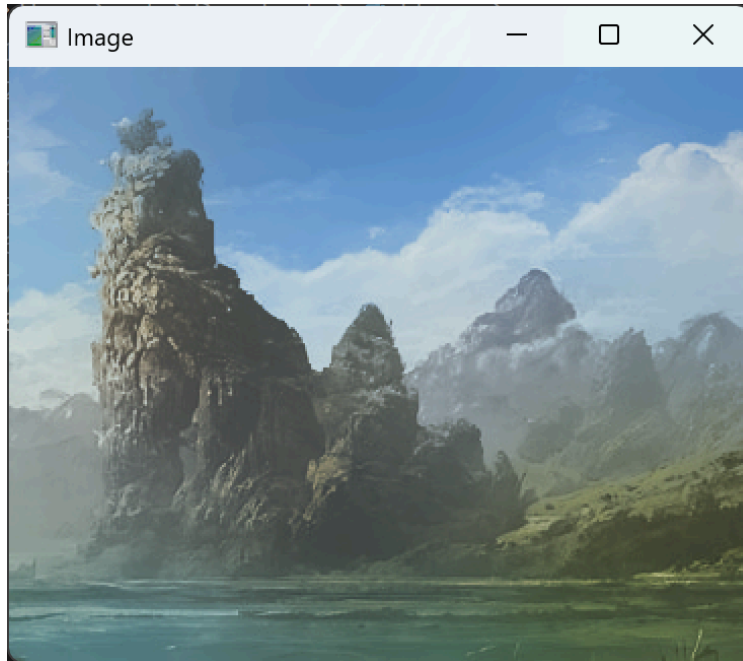
- Better than Gaussian Blur
- Helps in preserving edges.
- Edges are defined as the set of pixels forming a transition layer between large sets of pixels of same intensities. Basically areas of intense transition.

```
import cv2 as cv                # Import OpenCV (aliased as cv) for image pro
cessing
import numpy as np              # Import NumPy (commonly used with image
arrays)

img = cv.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path

blur = cv.bilateralFilter(img, d=100, sigmaColor=75, sigmaSpace=50)
# Apply bilateral filtering to the image
# d=100 sets the diameter of each pixel neighborhood
# sigmaColor=75 controls filtering based on color differences
# sigmaSpace=50 controls filtering based on spatial distance
```

```
cv.imshow("Image", blur)          # Display the filtered image in a window named "Image"
cv.waitKey(0)                     # Wait indefinitely until a key is pressed
cv.destroyAllWindows()           # Close all OpenCV windows
```



Edge Detection-

- Process of finding edges in an image. Edges are defined above.

Methods of Edge Detection -

Sobel Method -

- Contains two two kernels - SobelX (Gx) and SobelY(Gy).
- SobelX performs the task of gradient in X direction.
- SobelY performs the task of gradient in Y direction.
- $G = \sqrt{G_x^2 + G_y^2}$

Task: Using Sobel Method, detect the edges of any image.

```
import cv2 as cv                  # Import OpenCV (aliased as cv) for image processing
```

```

import numpy as np                # Import NumPy for numerical and array operations

img = cv.imread(r"C:\Users\jasti\Downloads\scenery.jpeg", cv.IMREAD_GRAYSCALE)
# Read the image in grayscale mode from the specified file path

blur = cv.GaussianBlur(img, (9, 9), 1.5)
# Apply Gaussian blur to reduce noise before edge detection
# (9, 9) is the kernel size, 1.5 is the standard deviation

sx = cv.Sobel(blur, ddepth=cv.CV_64F, dx=1, dy=0, ksize=3)
# Compute the Sobel gradient in the x-direction

sy = cv.Sobel(blur, ddepth=cv.CV_64F, dx=0, dy=1, ksize=3)
# Compute the Sobel gradient in the y-direction

mag = np.hypot(sx, sy)
# Compute gradient magnitude using  $\sqrt{sx^2 + sy^2}$ 

mag = mag / mag.max() * 255
# Normalize gradient magnitude to the range [0, 255]

mag = mag.astype(np.uint8)
# Convert gradient magnitude image to unsigned 8-bit format

abs_sx = np.absolute(sx)
abs_sy = np.absolute(sy)
# Take absolute values of Sobel gradients

abs_sx = (abs_sx / abs_sx.max() * 255).astype(np.uint8)
abs_sy = (abs_sy / abs_sy.max() * 255).astype(np.uint8)
# Normalize absolute Sobel gradients to [0, 255] and convert to uint8

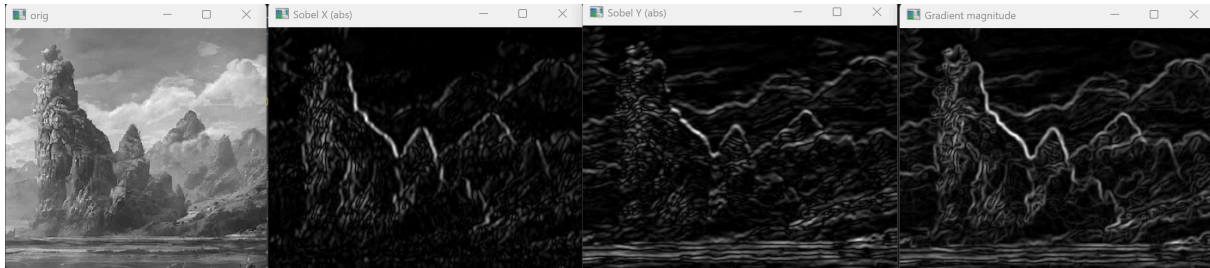
cv.namedWindow("orig", cv.WINDOW_NORMAL)
# Create a resizable window for the original image

cv.imshow("orig", img)            # Display the original grayscale image
cv.imshow("Sobel X (abs)", abs_sx) # Display absolute Sobel X gradient

```

```
cv.imshow("Sobel Y (abs)", abs_sy)    # Display absolute Sobel Y gradient
cv.imshow("Gradient magnitude", mag)  # Display gradient magnitude image

cv.waitKey(0)                        # Wait indefinitely until a key is pressed
cv.destroyAllWindows()               # Close all OpenCV windows
```



Laplace Method -

- Kernel form of Laplace Method -

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- This Kernel is equivalent to $\nabla^2 f = f_{xx} + f_{yy}$
- Comes from Taylor Series.
- Corresponds to second derivative.
- In Sobel's method, we see rate of change of intensity for edge. This does not work every time. For example, a canvas with same color whose intensity is changing constantly.

Task: Using Laplace Method, detect the edges of any image.

```
import cv2                                # Import OpenCV for image processing
import numpy as np                        # Import NumPy for numerical operations

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")
# Read the image from the specified file path

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Convert the image from BGR color space to grayscale
```

```

lap = cv2.Laplacian(gray, cv2.CV_64F)
# Apply the Laplacian operator to detect edges (second-order derivatives)

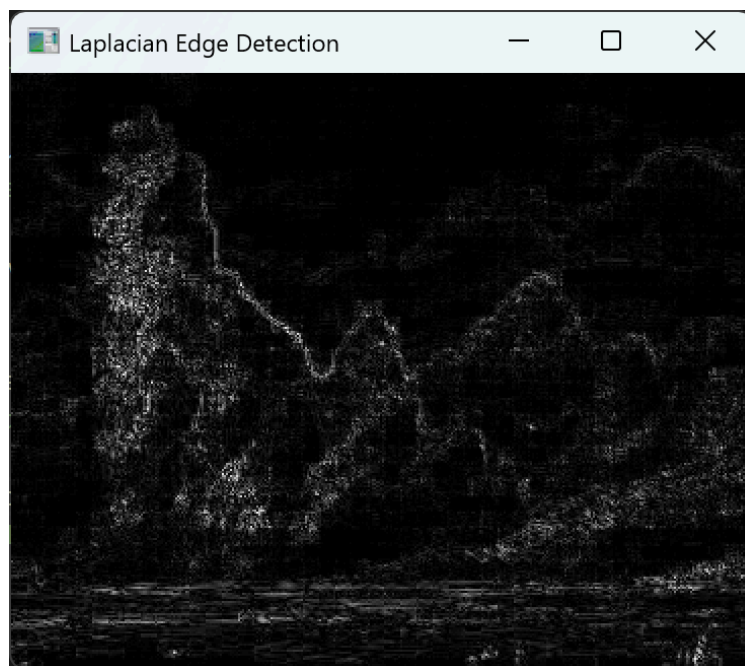
lap = np.uint8(np.absolute(lap))
# Take the absolute value of the Laplacian result and convert it to uint8 for display

cv2.imshow("Laplacian Edge Detection", lap)
# Display the Laplacian edge-detected image

cv2.waitKey(0)
# Wait indefinitely until a key is pressed

cv2.destroyAllWindows()
# Close all OpenCV windows

```



Gradients -

- $\nabla I = \frac{\delta I}{\delta x} \hat{x} + \frac{\delta I}{\delta y} \hat{y}$
- Assume pixels are as shown below

$$\begin{array}{cc}
 (x, y) & (1, 0) \\
 (0, 1) & (1, 1)
 \end{array}$$

- $\text{grad } I_x(x, y) = f(x + 1, y) - f(x, y)$
- $\text{grad } I_y(x, y) = f(x, y + 1) - f(x, y)$
- In order to avoid this, since these aren't really accurate, we use Sobel method to find gradient.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- $G = \sqrt{G_x^2 + G_y^2}$

Canny Edge Detection -

- Steps involved -
 1. Perform noise reduction using Gaussian blur.
 2. Apply Sobel's method to find out gradient.
 3. Perform Non-max oppression
 - Keep two threshold values T_0 and T_1 .
 - Detect Strong edge points and weak edge points
- Some NumPy functions that are helpful -
 1. $np.hypot(I_x, I_y)$
 2. $G = G/G.max() * 255$
 3. $theta = np.arctan2(I_y, I_x)$

Task: Using Canny Edge Detection, detect edges of any image

```
import cv2                                # Import OpenCV for image processing

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg", cv2.IMREAD_GRAYSCALE)
# Read the image from the specified file path in grayscale mode
```

```

img = cv2.resize(img, (732, 586))
# Resize the grayscale image to width = 732 pixels and height = 586 pixels

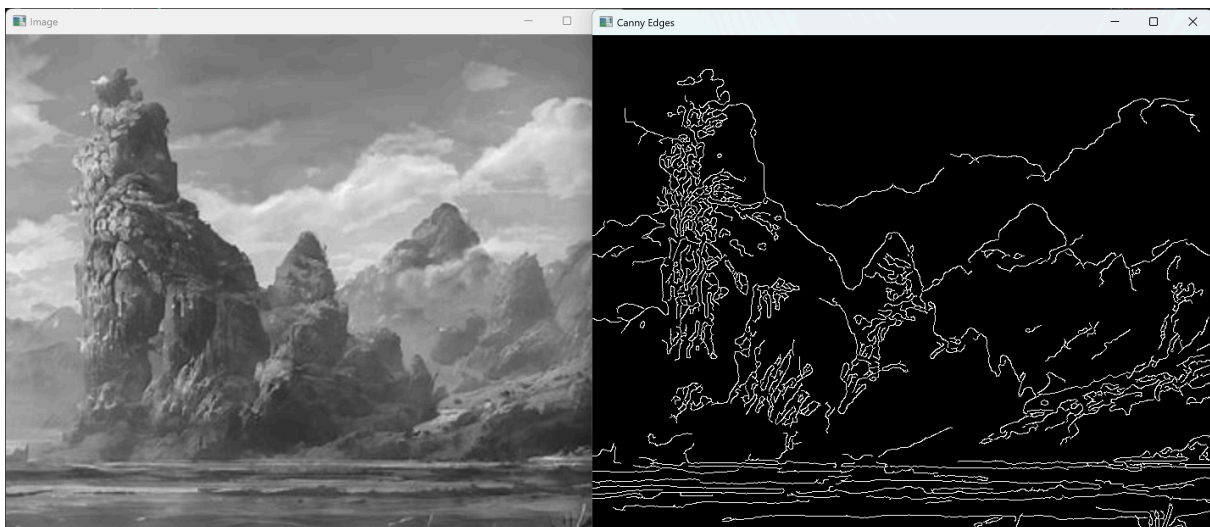
blurred = cv2.GaussianBlur(img, (5, 5), 1.4)
# Apply Gaussian blur to reduce noise before edge detection
# (5, 5) is the kernel size and 1.4 is the standard deviation

edges = cv2.Canny(blurred, 20, 100)
# Perform Canny edge detection
# 20 is the lower threshold and 100 is the upper threshold

cv2.imshow('Image', img)          # Display the original grayscale image
cv2.imshow('Canny Edges', edges)  # Display the detected edges

cv2.waitKey(0)                   # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()          # Close all OpenCV windows

```



Hough Transform -

- Sometimes edges are broken. To prevent those broken edges we use Hough Transform.
- Basic Principle -
 - $y = mx + c$
 - $c = -mx + y$
 - $\implies$ Every point in the XY plane corresponds to a line in the CM plane.

- $\implies$ If there are two points (x_1, y_1) and (x_2, y_2) , the intersection point of these two lines in CM plane will correspond to the line in the XY plane containing the two points.
- Initialize an ACCUMULATOR array.

Day 4 - Session 1

Machine Learning for Computer Vision

- We write general algorithms and feed data into it such that the computer learns from the data.
- Prediction function is a linear combination of parameters

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

θ s are called parameters

x_i s are called features

- Loss function -

We need to make the output of $h(x)$ as close to the actual value of the target variable.

$$J(\theta) = \frac{1}{2m} \sum_i (h_{\theta}(x)^i - y^i)^2$$

Linear regression is designed to reduce the value of the loss function.

Optimizing parameters -

- Find the optimal values of theta for which loss is minimized.
- 2 Methods are used -
 1. Least Squared Method
 2. Gradient Descent Method

Gradient Descent Method -

- Δx depends on $\frac{dy}{dx}$
- $x_{t+1} - x_t = -\alpha \frac{dy}{dx}$
- In terms of Machine Learning, y is Loss function
- x is theta.

$$\implies \theta_{t+1} = \theta_t - \alpha \frac{dL}{d\theta}$$

- $\frac{dL}{d\theta_1} = \frac{1}{2m} \sum_i 2x_1(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$
- α is called the Learning Rate.

Task: Write the code for linear regression using gradient descent method on the California Housing Dataset.

```
from sklearn.model_selection import train_test_split
import numpy as np
from numpy.linalg import inv

data = np.genfromtxt("housing.csv", delimiter=";", skip_header=1)
X = data[:, :-1]
y = data[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42
)

class LinearRegressionGDloops:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.theta = None
        self.mse_history_train = []
        self.mse_history_val = []

    def fit(self, X_train, y_train, X_test, y_test):
        # Add intercept term to X
        X_train_b = self.add_intercept(X_train)
        X_test_b = self.add_intercept(X_test)
        m, n = X_train_b.shape
        self.theta = np.zeros(n)

        # Gradient Descent
        for _ in range(self.iterations):
            predictions_train = self.predict(X_train_b)
            gradients = (2/m)* X_train_b.T.dot(predictions_train - y_train)
            self.theta -= self.learning_rate*gradients
```

```

# Compute and store MSE for training and validation sets
mse_train = self.compute_mse(y_train, predictions_train)
self.mse_history_train.append(mse_train)

predictions_test = self.predict(X_test_b)
mse_test = self.compute_mse(y_test, predictions_test)
self.mse_history_val.append(mse_test)

def predict(self, X):
    y_pred = []
    for i in range(X.shape[0]):
        y_pred.append(np.dot(X[i, :], self.theta))
    return np.array(y_pred)

def compute_mse(self, y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def add_intercept(self, X):
    return np.c_[np.ones(X.shape[0]), X] # Prepends a column of ones to the
array X, often used to add an intercept term to the feature matrix.

# Helper method to get the history for plotting
def get_mse_history(self):
    return [self.mse_history_train, self.mse_history_val]

# Compute parameters using the normal equation
def normal_equation(X, y):
    X_b = np.c_[np.ones(X.shape[0]), X] # Add intercept term
    return inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)

# Create and fit the model
model = LinearRegressionGDloops(learning_rate=0.01, iterations=1000)
model.fit(X_train, y_train, X_test, y_test)

```

Task: Write a program to implement Gradient Descent on a Simple Dataset and find the best fit line

```

import numpy as np          # Import NumPy for numerical computations
import matplotlib.pyplot as plt  # Import Matplotlib for plotting graphs

X = np.array([1, 2, 3, 4, 5])  # Input feature array (e.g., hours studied)
y = np.array([1.5, 3.7, 3.2, 4.8, 6.1]) # Target values (e.g., exam scores)

m = 0.0                    # Initialize slope (weight) of the linear model
b = 0.0                    # Initialize intercept (bias) of the linear model
lr = 0.01                  # Learning rate for gradient descent
epochs = 100               # Number of training iterations

for i in range(epochs):    # Loop for gradient descent optimization
    y_pred = m * X + b     # Predict values using the current model
    error = y_pred - y     # Compute prediction error

    dm = (2/len(X)) * np.dot(error, X) # Gradient of loss w.r.t slope (m)
    db = (2/len(X)) * np.sum(error)    # Gradient of loss w.r.t intercept (b)

    m -= lr * dm           # Update slope using gradient descent
    b -= lr * db           # Update intercept using gradient descent

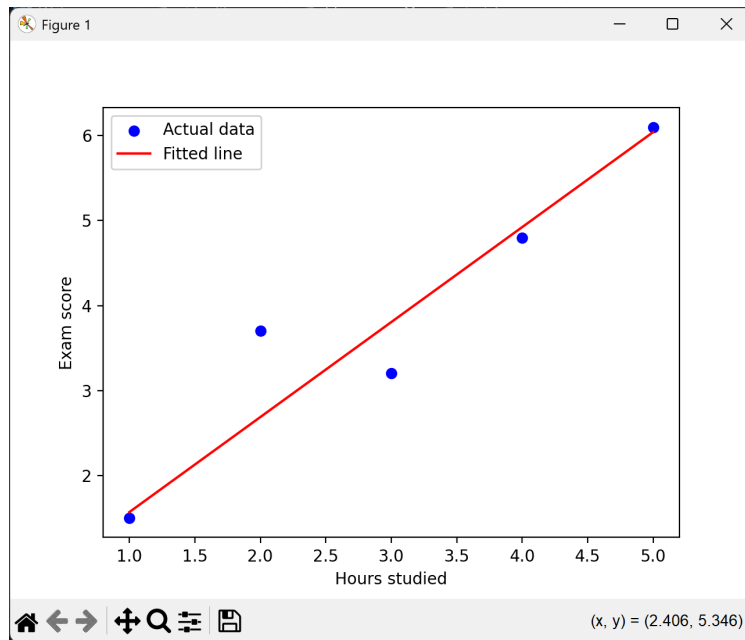
print(f"\nFinal model: y = {m:.2f}x + {b:.2f}")
# Print the final learned linear equation

plt.scatter(X, y, color='blue', label="Actual data")
# Plot the actual data points

plt.plot(X, m * X + b, color='red', label="Fitted line")
# Plot the fitted regression line

plt.xlabel("Hours studied")    # Label the x-axis
plt.ylabel("Exam score")      # Label the y-axis
plt.legend()                  # Show legend
plt.show()                    # Display the plot

```



Day 4 - Session 2

- Representation of Vectors as Matrices was introduced.
- Topic of lecture: Logistic Regression

Logistic Regression

- Fits the output between zero and one.
- Special function is used, known as sigmoid function.

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- Hence we define the following

$$\hat{y} = \frac{1}{1+e^{-(wx+b)}}$$

$$\frac{\partial \hat{y}}{\partial w} = \frac{\hat{y}(1-\hat{y})}{1}$$

Task: Write a program to implement Gradient Descent on a Simple Dataset for Logistic Regression.

```
import numpy as np          # Import NumPy for numerical array operations
import math                 # Import math module for mathematical functions
```

s

```
X = np.array([1, 2, 3, 4, 5])      # Input feature array
y = np.array([1.5, 3.7, 3.2, 4.8, 6.1]) # Original target values (continuous)

y_bin = np.array([1 if v > 3 else 0 for v in y], dtype=float)
# Convert the target values into binary labels:
# 1 if value > 3, else 0 (binary classification)

w = 0.0          # Initialize weight
b = 0.0          # Initialize bias
lr = 0.1         # Learning rate

for _ in range(1000):      # Run gradient descent for 1000 iterations
    dw = 0.0              # Gradient of loss w.r.t weight
    db = 0.0              # Gradient of loss w.r.t bias

    for i in range(len(X)): # Loop over each training sample
        z = w * X[i] + b    # Linear combination
        pred = 1 / (1 + math.exp(-z)) # Sigmoid activation (prediction)
        dw += (pred - y_bin[i]) * X[i] # Accumulate gradient for weight
        db += (pred - y_bin[i])       # Accumulate gradient for bias

    w -= lr * dw / len(X)      # Update weight using average gradient
    b -= lr * db / len(X)      # Update bias using average gradient

print(round(w, 4), round(b, 4))
# Print the trained weight and bias rounded to 4 decimal places

predictions = []             # List to store predicted probabilities
for i in range(len(X)):
    z = w * X[i] + b          # Linear combination for each input
    predictions.append(round(1 / (1 + math.exp(-z)), 4))
    # Compute and store sigmoid output rounded to 4 decimal places

print(predictions)           # Print the predicted probabilities
```

2.717 -3.7029
[0.2717, 0.8496, 0.9884, 0.9992, 0.9999]

Universal Approximation Theorem

- **Claim:** Circle $\approx$ Triangle
- Call a Triangle an approximation of a circle.

Neural Networks

- Uses composition of Logistic sigmoid function several times to approximate the curve to the best fit model to perform an operation on a dataset.
- Activation functions -
 1. $\sigma(x)$
 2. $\tanh(x)$
 3. $ReLU(x)$

$$ReLU(x) = \max(0, x) = \begin{cases} x, & 0 \leq x \\ 0, & x \leq 0 \end{cases}$$

- Problem with $ReLU(x)$ is that once the value becomes negative it's impossible to go in the other direction.
- Chain Rule is very important in Neural networks.
- Sometimes we store some derivative values in variables to reduce computational power.
- Read about vanishing/exploding gradient problem
- To fix the ReLU Problem, engineers came up with a function called Leaky ReLU function.
 - $Leaky\ ReLU(x) = \max(\epsilon x, x)$, where ϵ is a very small value.
- Neural Networks are called Fully Connected Layers (FCs in short)

Convolutional Neural Networks

Parameters and Hyperparameters

- Parameters include weights and biases.
- Hyper Parameters include Output size, Number of Hidden Layers, Size of hidden layers

Introduction of a new Loss function -

$$\text{Cross Entropy Loss} = \sum_i [y_i \log \hat{y}_i]$$

Kernels Continued

- Always choose the dimensions of Kernels to be odd dimensional.
- When a kernel slides onto an image of dimension I, dimension of output image will be

$$Dim = \frac{I - K + 2P}{S} + 1$$

Where P is the Padding

S is the Stride

K is dimension of Kernel

I is dimension of Image

- Softmax →

$$\phi(y_i) = \frac{e^{\hat{y}_i}}{\sum_i^n e^{\hat{y}_i}}$$

- Overfitting: Process of models memorizing the data-set instead of getting trained.
 - Training accuracy is almost 100%.
 - Testing accuracy is around 25%.

Training Models

Gradient Descent

- Batch Gradient
- Stochastic

- Batch Stochastic
- Momentum
- Nesterov Accelerated
- Adam
- AdaGrad

Day 5 - Session 1

How to Represent Graphs

1. Edge List
2. Adjacency List
3. Adjacency Matrix

Edge List -

- Consists of Nodes and Edges.
- Stores tuples of the form (Start node, End node, Weight)

Adjacency List -

- Array of size V , in which each Index holds a linked list containing vertices that the vertex at that index is connected to.
- Most popular representation.
- Eg. If node 0 has 1 and 2 as neighbors, then index 0 would contain [1, 2]

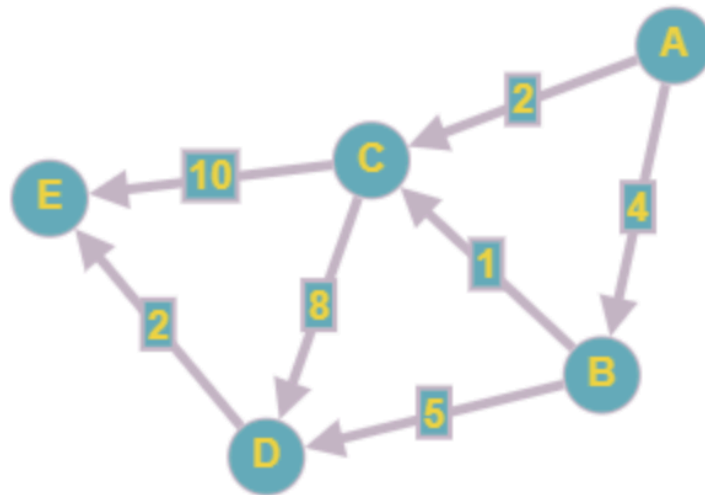
Adjacency Matrix -

- Contains only 0 and 1 for unweighted graph.
- Contains positive integers for weighted graph.
- Rows contain start nodes, Columns are destination nodes.
- Non-directional Graph $\implies$ Symmetric Matrix
- Directional Graph $\implies$ Unsymmetric Matrix

Graph Traversal -

- Systematic process of visiting all vertices of a graph in a particular order.

Graph to be used for BFS, DFS and Dijkstra's Algorithm Implementation



Edge List Representation of Above Graph

```
graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2], ['C', 'E', 10]]
```

Breadth First Search -

- Check Level by level.
- Explore widely before going deep.

Algorithm -

1. Mark all vertices as unvisited.
2. Create an empty queue Q and enqueue s. Mark s as under-visit.
3. While Q ≠ empty:
 - (i) Dequeue a vertex u from Q and mark u visited.
 - (ii) For each unvisited neighbor v of u:
 - (a) Mark v as under-visit, set u as its parent, and enqueue v.

Task: Write a general code to implement BFS algorithm

```

graph = {
    'A': [],
    'B': [],
    'C': [],
    'D': [],
    'E': [],
    'F': []
}
# Define a graph using an adjacency list representation
# Each key is a node, and the value is a list of connected neighbors

start = 'A'
# Define the starting node for BFS traversal

visited = set()
# Set to keep track of visited nodes

queue = [start]
# Initialize the queue with the starting node

while queue:                                # Continue traversal while the queue is not empty
    node = queue.pop(0)                      # Remove the first element from the queue (FIFO)

    if node not in visited:
        print(node)                         # Print the current node
        visited.add(node)                   # Mark the node as visited

        for n in graph[node]:               # Iterate over all neighbors of the current node
            if n not in visited:
                queue.append(n)              # Add unvisited neighbors to the queue

```

Task: Implement BFS Algorithm on the given graph.

```

graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2],
['C', 'E', 10]]

```

```

# Define the graph as a list of edges with weights (u, v, weight)

adj = {}
# Initialize an empty adjacency list dictionary

for e in graph:
    u = e[0]          # Source node of the edge
    v = e[1]          # Destination node of the edge
    if u not in adj:
        adj[u] = []   # Create a list for the node if it doesn't exist
    adj[u].append(v)  # Add the neighbor to the adjacency list

start = 'A'
# Define the starting node for BFS

visited = []
queue = []
# Initialize visited list and queue for BFS traversal

visited.append(start)    # Mark the start node as visited
queue.append(start)      # Add the start node to the queue

while queue:
    node = queue.pop(0)   # Remove the first element from the queue
    (FIFO)
    print("Popped:", node) # Print the node being processed

    if node in adj:
        for n in adj[node]: # Traverse all neighbors of the current node
            if n not in visited:
                visited.append(n) # Mark neighbor as visited
                queue.append(n)  # Add neighbor to the queue

print(visited)           # Print the order of visited nodes
print(adj)               # Print the adjacency list

```

```
Popped: A
Popped: B
Popped: C
Popped: D
Popped: E
['A', 'B', 'C', 'D', 'E']
{'A': ['B', 'C'], 'B': ['C', 'D'], 'C': ['D', 'E'], 'D': ['E']}
```

Depth First Search -

- Keep moving until you hit a wall.
- Backtrack when you hit a dead-end or visited node.

Algorithm -

1. Mark all vertices as unvisited
2. Create an empty stack S and push s; marks s as under-visit
3. while S≠empty do
 - Let u be the top of S
 - if u has an unvisited neighbor v then: //Visit v along the edge (u,v)
 - Set $\pi[v]$ as u, mark v as under-visit, and push v onto S
 - else //Backtrack to parent u along the edge (v,u)
 - Pop u from S and mark it visited

Task: Write a general code to implement DFS algorithm

```
graph = {
    'A': [],
    'B': [],
    'C': [],
    'D': [],
    'E': [],
    'F': []
}
```

```
start = 'A'
```

```
visited = set()
```

```

stack = [start]

while stack:
    node = stack.pop()
    if node not in visited:
        print(node)
        visited.add(node)
        for n in reversed(graph[node]):
            if n not in visited:
                stack.append(n)

```

Task: Implement DFS Algorithm on the given graph.

```

graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2],
['C', 'E', 10]]
# Define the graph as a list of edges with weights (u, v, weight)

adj = {}
# Initialize an empty adjacency list dictionary

for e in graph:
    u = e[0]                # Source node of the edge
    v = e[1]                # Destination node of the edge
    if u not in adj:
        adj[u] = []        # Create a list for the node if it doesn't exist
    adj[u].append(v)        # Add the neighbor to the adjacency list

visited = []
# List to keep track of visited nodes during DFS

def dfs(node):
    visited.append(node)    # Mark the current node as visited
    print(node)            # Print the current node

    if node in adj:
        for n in adj[node]: # Traverse all neighbors of the current node
            if n not in visited:
                dfs(n)       # Recursively visit unvisited neighbors

```

```
dfs('A')          # Start DFS traversal from node 'A'
print(adj)         # Print the adjacency list
```

```
A
B
C
D
E
{'A': ['B', 'C'], 'B': ['C', 'D'], 'C': ['D', 'E'], 'D': ['E']}
```

Dijkstra's Algorithm

- Greedy Algorithm.
- Assign every node a tentative distance.
 - Start Node $\rightarrow 0$
 - All other Nodes $\rightarrow \infty$
- Maintain set of unvisited nodes.
- Choose unvisited node with shortest distance.
- Mark it visited $\rightarrow$ its shortest distance is now final.
- Relax all outgoing edges from that node: $d[v] = \min(d[v], d[u] + w(u, v))$

Algorithm -

Initialization -

1. For all vertices $v \in V$:
 - $\text{dist}[v] = \infty$
 - $\text{parent}[v] = \text{null}$
2. Set the source vertex:
 - $\text{dist}[s] = 0$
3. Insert the source into the priority queue:
 - push $(0, s)$ into priority queue

Main Loop -

While the priority queue is **not empty**:

1. Extract the node with minimum distance:

$(du, u) = \text{extract_min}()$

2. If u is already visited:
 - Continue to the next iteration
3. Mark u as visited
4. For each neighbor v of u :
 - If a shorter path is found:

```
if dist[u] + w(u, v) < dist[v]:
    dist[v] = dist[u] + w(u, v)
    parent[v] = u
    push (dist[v], v)
```

Outputs -

- $\text{dist}[v] \rightarrow$ Shortest distance from source s to vertex v
- $\text{parent}[v] \rightarrow$ Used to reconstruct the shortest path

Task: Implement Dijkstra's Algorithm on the given graph.

```
graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2],
['C', 'E', 10]]
# Define the graph as a list of edges: (source, destination, weight)

adj = {}
# Dictionary to store the adjacency list with weights

for u, v, w in graph:
    if u not in adj:
        adj[u] = []          # Initialize adjacency list for node u
    adj[u].append((v, w))   # Add (neighbor, weight) to adjacency list

dist = {}
prev = {}
nodes = []
# dist: stores shortest distance from source to each node
# prev: stores previous node in the shortest path
# nodes: list of all unique nodes
```

```

for e in graph:
    dist[e[0]] = 9999          # Initialize distance for source node
    dist[e[1]] = 9999          # Initialize distance for destination node
    prev[e[0]] = None          # Initialize previous node for source
    prev[e[1]] = None          # Initialize previous node for destination
    if e[0] not in nodes:
        nodes.append(e[0])     # Add source node if not already present
    if e[1] not in nodes:
        nodes.append(e[1])     # Add destination node if not already present

dist['A'] = 0
# Set distance of the start node 'A' to 0

while nodes:                  # Dijkstra's algorithm main loop
    u = nodes[0]
    for n in nodes:
        if dist[n] < dist[u]:
            u = n              # Select node with minimum distance
    nodes.remove(u)            # Remove the selected node from unvisited nodes

    if u in adj:
        for v, w in adj[u]:    # Relax edges from node u
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w # Update shortest distance to v
                prev[v] = u          # Update previous node for v

path = []
cur = 'E'
# Reconstruct the shortest path from 'A' to 'E'

while cur:
    path.insert(0, cur)        # Insert current node at the beginning of the path
    cur = prev[cur]            # Move to the previous node

print("Path:", path)           # Print the shortest path from A to E
print("Distance:", dist['E'])  # Print the total cost of the shortest path

```

```
Path: ['A', 'B', 'D', 'E']
Distance: 11
```

Day 5 - Session 2

SLAM

- Simultaneous Localization and Mapping
- Assume a Robot that only knows its start position and final destination but does not know the route to it.
- It considers errors that the robot comes across when using its LiDAR.

Task: Write the general code for A* Algorithm (When diagonal movement is not allowed)

```
grid = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,1,0,0],
    [0,0,0,0,0]
]
# Define the grid map
# 0 represents free space, 1 represents obstacles

start = (0, 0)           # Starting position (x, y)
goal = (4, 4)            # Goal position (x, y)

open_list = [start]      # List of nodes to be explored
came_from = {}           # Dictionary to reconstruct the path
g = {start: 0}           # Cost from start to each node
f = {start: abs(start[0]-goal[0]) + abs(start[1]-goal[1])}
# f-score = g-score + heuristic (Manhattan distance)

def neighbors(n):
    x, y = n              # Current node coordinates
    res = []              # List to store valid neighbors
```

```

for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
    nx, ny = x+dx, y+dy      # Compute neighbor coordinates
    if 0 <= nx < 5 and 0 <= ny < 5 and grid[ny][nx] == 0:
        res.append((nx, ny))  # Add neighbor if inside grid and not an obstacle
    return res

while open_list:              # Continue until there are nodes to explore
    cur = open_list[0]         # Initialize current node
    for n in open_list:
        if f[n] < f[cur]:
            cur = n             # Select node with lowest f-score

    if cur == goal:
        break                   # Stop search when goal is reached

    open_list.remove(cur)      # Remove current node from open list

    for n in neighbors(cur):    # Explore all valid neighbors
        temp = g[cur] + 1      # Tentative g-score for neighbor
        if n not in g or temp < g[n]:
            came_from[n] = cur  # Record best path to neighbor
            g[n] = temp         # Update g-score
            f[n] = g[n] + abs(n[0]-goal[0]) + abs(n[1]-goal[1])
            # Update f-score using Manhattan heuristic

        if n not in open_list:
            open_list.append(n) # Add neighbor to open list if not present

path = []
cur = goal                    # Start path reconstruction from goal
while cur in came_from:
    path.insert(0, cur)        # Insert node at the beginning of the path
    cur = came_from[cur]
path.insert(0, start)         # Add the start node to the path

dist = len(path) - 1          # Compute path length (number of steps)

```

```
print("Path:", path)          # Print the final path
print("Path Length:", dist)   # Print the length of the path
```

```
Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Path Length: 8
```

Task: Write the general code for A* Algorithm (When diagonal movement is allowed)

```
grid = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,1,0,0],
    [0,0,0,0,0]
]
# Define a 5x5 grid
# 0 represents free space, 1 represents obstacles

start = (0, 0)          # Starting position (x, y)
goal = (4, 4)           # Goal position (x, y)

open_list = [start]     # List of nodes to be evaluated
came_from = {}          # Dictionary to reconstruct the path
g = {start: 0}           # Cost from start to each node
f = {start: abs(start[0]-goal[0]) + abs(start[1]-goal[1])}
# f-score = g-score + heuristic (Manhattan distance)

def neighbors(n):
    x, y = n             # Current node coordinates
    res = []             # List to store valid neighbors
    for dx, dy in [(1,0),(-1,0),(0,1),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]:
        # Allow movement in 8 directions (including diagonals)
        nx, ny = x+dx, y+dy    # Neighbor coordinates
        if 0 <= nx < 5 and 0 <= ny < 5 and grid[ny][nx] == 0:
            res.append((nx, ny)) # Add neighbor if inside grid and not blocked
    return res

while open_list:         # Continue until there are nodes to explore
```

```

cur = open_list[0]          # Initialize current node
for n in open_list:
    if f[n] < f[cur]:
        cur = n            # Select node with the lowest f-score

if cur == goal:
    break                  # Stop search if goal is reached

open_list.remove(cur)      # Remove current node from open list

for n in neighbors(cur):   # Explore all valid neighbors
    # Determine step cost: diagonal moves cost 1.4, straight moves cost 1
    if abs(n[0]-cur[0]) + abs(n[1]-cur[1]) == 2:
        step = 1.4
    else:
        step = 1

    temp = g[cur] + step    # Tentative cost to reach neighbor

    if n not in g or temp < g[n]:
        came_from[n] = cur  # Update best parent for neighbor
        g[n] = temp         # Update g-score
        f[n] = g[n] + abs(n[0]-goal[0]) + abs(n[1]-goal[1])
        # Update f-score using Manhattan heuristic

    if n not in open_list:
        open_list.append(n) # Add neighbor to open list

path = []
cur = goal
dist = 0                    # Total path length

while cur in came_from:    # Reconstruct path from goal to start
    p = came_from[cur]

    # Accumulate distance based on move type
    if abs(p[0]-cur[0]) + abs(p[1]-cur[1]) == 2:
        dist += 1.4
    else:

```

```

        dist += 1

    path.insert(0, cur)          # Insert current node at the beginning of the path
    cur = p

    path.insert(0, start)        # Add the start node to the path

    print("Path:", path)         # Print the final path
    print("Path Length:", dist)  # Print the total path length

```

```

Path: [(0, 0), (1, 0), (2, 1), (2, 2), (3, 3), (4, 4)]
Path Length: 6.199999999999999

```

Task: Take an image, convert it to black and white and detect the edges using Canny Edge Detection. Pick two points and implement A* Algorithm to find the optimal path between the points. Consider the edges to be obstacles.

```

import cv2                                # Import OpenCV for image processing and display
import numpy as np                        # Import NumPy for numerical and array operations

img = cv2.imread(r"C:\Users\jasti\Downloads\maze_Astar.png")
# Read the maze image from the specified file path

img = cv2.resize(img, (400, 400))
# Resize the image to a fixed size of 400×400 pixels

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Convert the image to grayscale

edges = cv2.Canny(gray, 50, 150)
# Detect edges using the Canny edge detector

ys, xs = np.where(edges > 0)
# Get coordinates of edge pixels (obstacles)

```

```

min_x, max_x = xs.min(), xs.max()
min_y, max_y = ys.min(), ys.max()
# Determine the bounding box of the detected edges

mask = np.ones_like(edges) * 255
# Create a white mask image

mask[min_y:max_y, min_x:max_x] = edges[min_y:max_y, min_x:max_x]
# Keep edges only inside the bounding box and block everything else

edges = mask
# Update edges image with masked result

grid = edges / 255
# Convert edges image into a grid (0 = free, 1 = obstacle)

start = (30, 30)           # Define the start position
end = (380, 380)          # Define the end position

open_list = [(0, start)]   # Open list containing (f_score, node)
came_from = {}             # Dictionary to reconstruct the path
g_score = {start: 0}       # Cost from start to each node

while open_list:           # Continue until there are nodes to explore
    open_list.sort()        # Sort nodes by lowest f-score
    current = open_list.pop(0)[1] # Select the node with the lowest f-score

    if current == end:
        break               # Exit loop if the goal is reached

    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        # Explore 4-connected neighbors (up, down, right, left)

        neighbor = (current[0] + dx, current[1] + dy)

        if 0 <= neighbor[0] < 400 and 0 <= neighbor[1] < 400:
            # Check if neighbor is within image boundaries

```

```

    if grid[neighbor[1], neighbor[0]] == 1:
        continue          # Skip if neighbor is an obstacle

    temp_g = g_score[current] + 1
    # Tentative cost from start to neighbor

    if neighbor not in g_score or temp_g < g_score[neighbor]:
        # Update path if a shorter one is found
        g_score[neighbor] = temp_g

    h = abs(neighbor[0] - end[0]) + abs(neighbor[1] - end[1])
    # Manhattan distance heuristic

    f_score = temp_g + h
    # Total estimated cost

    open_list.append((f_score, neighbor))
    came_from[neighbor] = current
    # Record the path

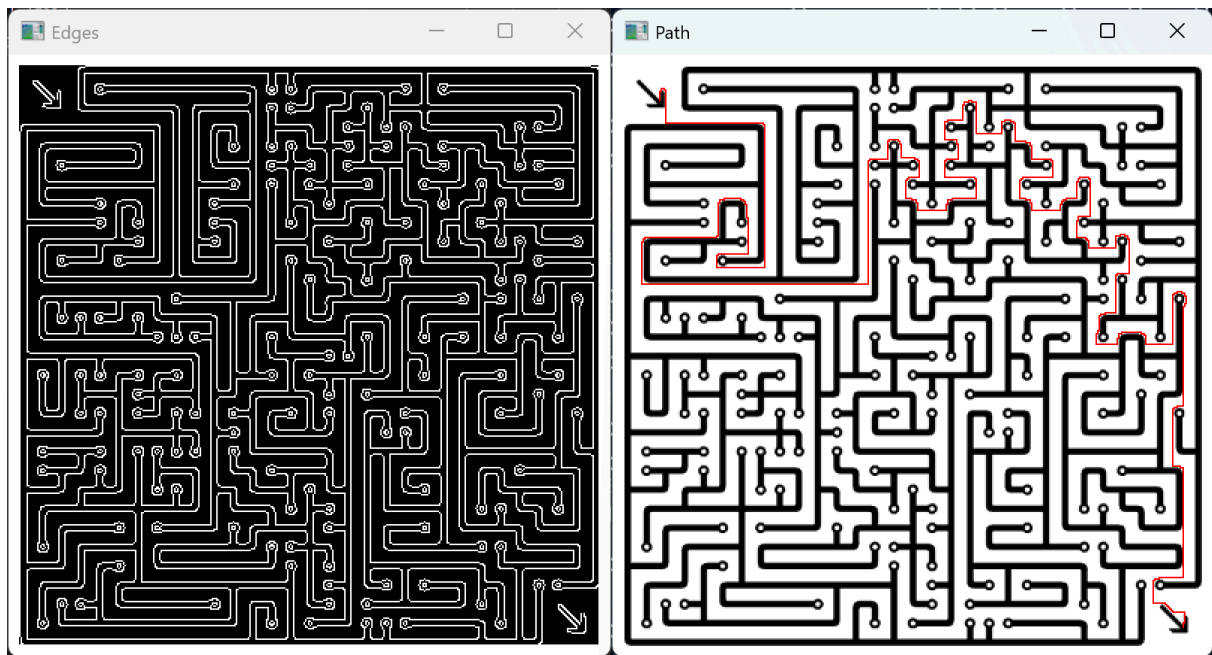
curr = end
while curr in came_from:
    img[curr[1], curr[0]] = [0, 0, 255]
    # Draw the final path in red on the original image
    curr = came_from[curr]

cv2.imshow('Edges', edges)
# Display the processed edge map with blocked regions

cv2.imshow('Path', img)
# Display the maze image with the A* path drawn

cv2.waitKey(0)          # Wait indefinitely until a key is pressed
cv2.destroyAllWindows() # Close all OpenCV windows

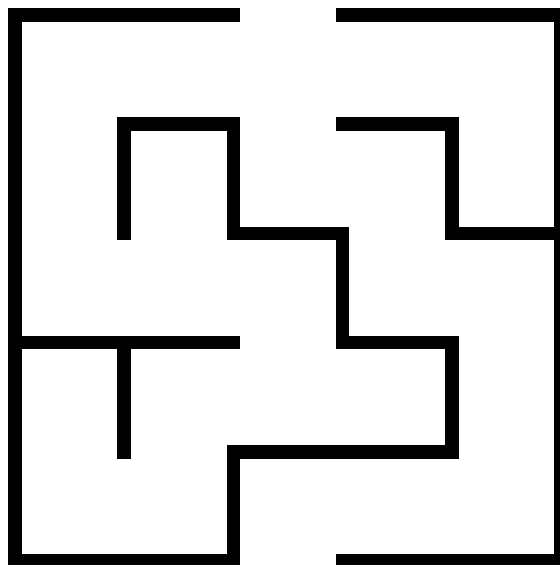
```



Day 6 - Session 1

- Computationally expensive to implement A* on image.
- Therefore we don't consider every pixel as a node. We try to find different optimal nodes on the image we are dealing, instead of all pixels.

Perform the following tasks to implement algorithms on this given maze.



Probabilistic Roadmap (PRM)

- Configurational space, free space and obstacles are introduced.
- Nodes must be on the free space.
- Generate a random node, and check if it lies in the free space. If in free space, accepted. If it intersects with obstacle, reject and generate a new node.
- Add accepted nodes to the graph. Connect every new node to its k closest nodes.
- The value of k must be optimal, neither too high nor too less.
- If any edge is intersecting with obstacles, discard the edge.
- This will create a graph. Then apply A* on it.

Task: Implement PRM on the given maze.

```
import cv2                # Import OpenCV for image processing and display
import numpy as np        # Import NumPy for numerical and array operations
import random             # Import random module for random sampling

img = cv2.imread(r"C:\Users\jasti\Downloads\maze.png")
# Read the maze image from the specified file path

img = cv2.resize(img, (400, 400))
# Resize the image to 400×400 pixels

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Convert the image to grayscale

_, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
# Apply binary thresholding to obtain a free-space (white) and obstacle (black) map

start = (30, 370)         # Define the start node coordinates
end = (200, 395)          # Define the end node coordinates
nodes = [start, end]      # Initialize node list with start and end nodes

while len(nodes) < 100:    # Randomly sample nodes until total nodes =
```

```

100
    rx = random.randint(5, 395)      # Random x-coordinate within bounds
    ry = random.randint(5, 395)      # Random y-coordinate within bounds
    if thresh[ry, rx] == 255:
        nodes.append((rx, ry))      # Add node only if it lies in free space

adj = {i: [] for i in range(len(nodes))}
# Create an adjacency list for each node

for i in range(len(nodes)):          # Loop over all node pairs
    for j in range(i + 1, len(nodes)):
        p1 = nodes[i]
        p2 = nodes[j]

        dist = ((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)**0.5
        # Compute Euclidean distance between two nodes

        if dist < 100:
            # Consider connecting nodes only if within a distance threshold

            line_img = np.zeros((400, 400), dtype=np.uint8)
            # Create a blank image to test collision along the edge

            cv2.line(line_img, p1, p2, 255, 1)
            # Draw a line between the two nodes

            if not np.any(cv2.bitwise_and(line_img, cv2.bitwise_not(thresh))):
                # Check if the line intersects any obstacle
                adj[i].append(j)      # Add edge i → j
                adj[j].append(i)      # Add edge j → i (undirected graph)

queue = [(0, [0])]                   # Priority queue storing (distance, path)
visited = set()                      # Set to track visited nodes
path = []                           # Final path storage

while queue:                         # Dijkstra-style search through the roadmap
    queue.sort()                     # Sort queue by distance
    dist, current_path = queue.pop(0) # Extract the shortest path so far
    last_node = current_path[-1]      # Get the last node in the path

```

```

if last_node == 1:
    path = current_path          # Path found to the end node
    break

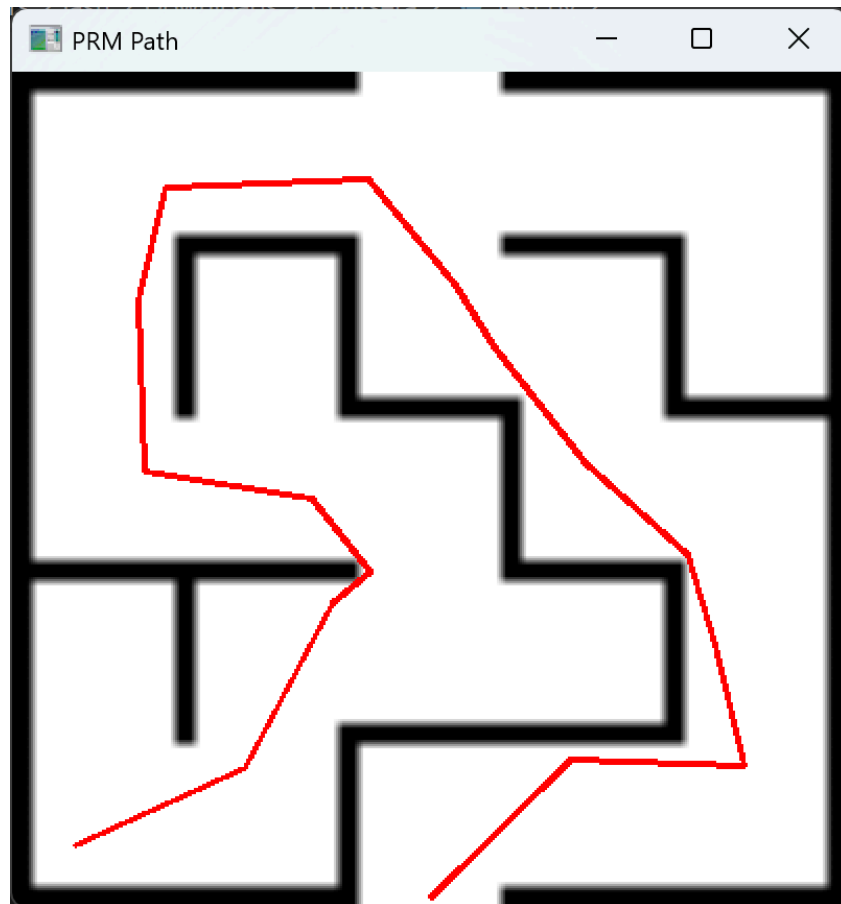
if last_node not in visited:
    visited.add(last_node)       # Mark node as visited
    for neighbor in adj[last_node]:
        # Explore all connected neighbors
        new_dist = dist + (
            (nodes[last_node][0]-nodes[neighbor][0])**2 +
            (nodes[last_node][1]-nodes[neighbor][1])**2
        )**0.5
        # Update distance with edge cost

        queue.append((new_dist, current_path + [neighbor]))
        # Push new path to the queue

for i in range(len(path)-1):
    cv2.line(img, nodes[path[i]], nodes[path[i+1]], (0, 0, 255), 2)
    # Draw the final path on the maze image in red

cv2.imshow('PRM Path', img)     # Display the maze with PRM path
cv2.waitKey(0)                  # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()         # Close all OpenCV windows

```



Rapidly-exploring Random Trees (RRT)

- Start from the source by adding a node there.
- Generate a point. Check if there are no obstacles between it and its nearest node. If distance is greater than THRESHOLD DISTANCE.
- If distance greater than threshold distance, shift point along the line such that distance equals threshold value.
- Now that's the new node. Check if there is an obstacle between this new node and the nearest node.
- If there is an obstacle, DISCARD THE NODE (not just the edge).
- Input to the computer are number of nodes to be generated and threshold distance.

Task: Implement RRT on the given maze.

```
import cv2                                # Import OpenCV for image processing and display
import numpy as np                        # Import NumPy for numerical and array operations
```

```

rations
import random                # Import random module for random sampling

img = cv2.imread(r"C:\Users\jasti\Downloads\maze.png")
# Read the maze image from the specified file path

img = cv2.resize(img, (400, 400))
# Resize the image to 400×400 pixels

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Convert the image to grayscale

_, bw = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
# Convert the grayscale image to a binary image (free space = white, obstacles = black)

start = (30, 370)            # Define the start point
goal = (200, 395)           # Define the goal point

nodes = [start]              # Initialize the tree with the start node
parent = {start: None}       # Dictionary to store parent of each node
step = 20                    # Step size for RRT expansion

def free(p):
    # Check whether a point lies in free space
    return bw[p[1], p[0]] == 255

def dist(a, b):
    # Compute Euclidean distance between two points
    return ((a[0]-b[0])**2 + (a[1]-b[1])**2) ** 0.5

def steer(a, b):
    # Steer from point a towards point b with maximum step size
    d = dist(a, b)
    if d < step:
        return b
    x = int(a[0] + step * (b[0]-a[0]) / d)
    y = int(a[1] + step * (b[1]-a[1]) / d)
    return (x, y)

```

```

def collision(a, b):
    # Check for collision along the straight line between a and b
    for i in range(21):
        x = int(a[0] + (b[0]-a[0]) * i / 20)
        y = int(a[1] + (b[1]-a[1]) * i / 20)
        if bw[y, x] == 0:
            return True
    return False

found = False                # Flag to indicate whether a path to the goal is found

for _ in range(5000):        # Run RRT expansion for a fixed number of iterations
    rand = (random.randint(0, 399), random.randint(0, 399))
    # Sample a random point in the image

    if not free(rand):
        continue             # Skip if the sampled point is not in free space

    near = nodes[0]           # Find the nearest existing node
    for n in nodes:
        if dist(n, rand) < dist(near, rand):
            near = n

    new = steer(near, rand)    # Generate a new node in the direction of the random point

    if free(new) and not collision(near, new):
        nodes.append(new)     # Add the new node to the tree
        parent[new] = near    # Store its parent

    if dist(new, goal) < step and not collision(new, goal):
        parent[goal] = new    # Connect the goal if it is reachable
        found = True
        break

if found:

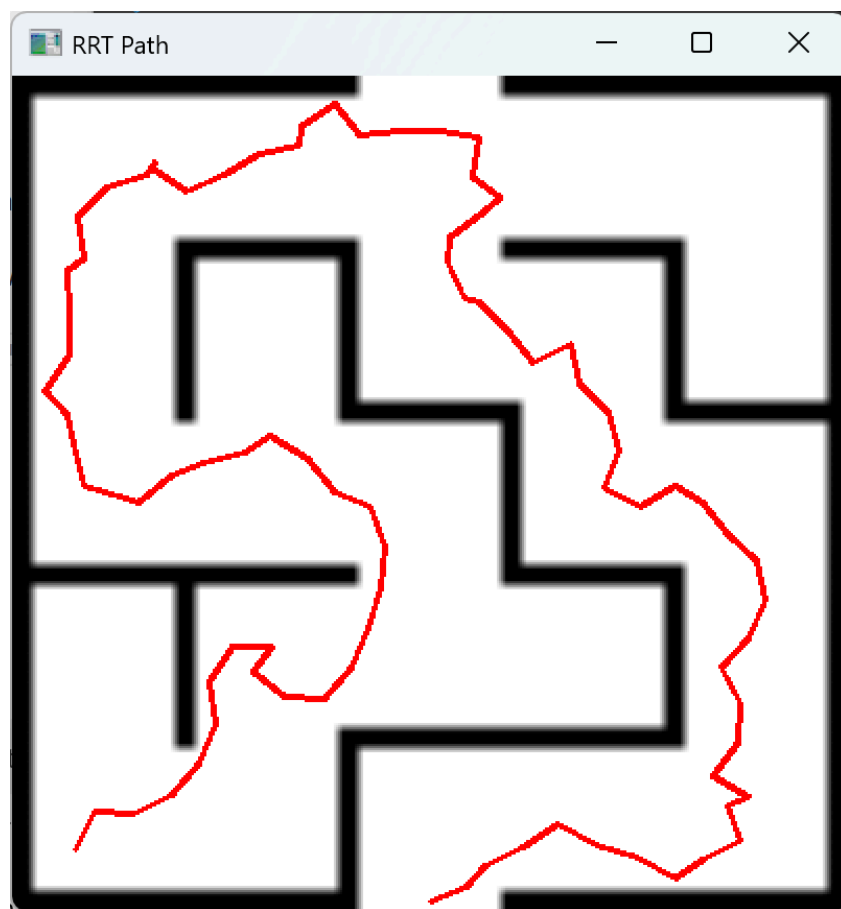
```

```

cur = goal                # Backtrack from goal to start
while parent[cur] is not None:
    p = parent[cur]
    cv2.line(img, p, cur, (0, 0, 255), 2)
    # Draw the path segment in red
    cur = p

cv2.imshow("RRT Path", img)    # Display the maze with the RRT path
cv2.waitKey(0)                # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()       # Close all OpenCV windows

```



Rapidly-exploring Random Trees Star (RRT*)

- Choose best parent instead of nearest parent.
- In RRT*, we also check the possibility assuming that THE NEWLY GENERATED NODE IS THE BEST PARENT OF ITS BEST PARENT.
- Eg, if nodes are A, B, C
 $A \rightarrow B \rightarrow C$

where C is newly generated node and B is best parent

- We check what happens in the case where $A \rightarrow C \rightarrow B$
- If the new cost is less than old cost, we consider this case on the above line.

Task: Implement RRT* on the given maze.

```
import cv2                                # Import OpenCV for image processing and display
import numpy as np                        # Import NumPy for numerical operations
import random                             # Import random module for random sampling
import math                               # Import math module for mathematical functions

img = cv2.imread(r"C:\Users\jasti\Downloads\maze.png")
# Read the maze image from the specified file path

img = cv2.resize(img, (400, 400))
# Resize the image to 400×400 pixels

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Convert the image to grayscale

_, bw = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
# Convert the grayscale image into a binary image (free space = white, obstacles = black)

start = (30, 370)                         # Define the start point
goal = (200, 395)                        # Define the goal point

nodes = [start]                          # Initialize the node list with the start node
parent = {start: None}                   # Dictionary to store parent relationships
cost = {start: 0}                        # Dictionary to store cost from start to each node

step = 20                                # Step size for tree expansion
radius = 40                              # Neighborhood radius for rewiring (RRT*)

def free(p):
```

```

# Check if a point is within bounds and lies in free space
return 0 <= p[0] < 400 and 0 <= p[1] < 400 and bw[p[1], p[0]] == 255

def dist(a, b):
    # Compute Euclidean distance between two points
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

def steer(a, b):
    # Move from point a towards point b by at most 'step' distance
    d = dist(a, b)
    if d < step:
        return b
    x = int(a[0] + step * (b[0]-a[0]) / d)
    y = int(a[1] + step * (b[1]-a[1]) / d)
    return (x, y)

def collision(a, b):
    # Check whether the straight path from a to b intersects any obstacle
    for i in range(21):
        x = int(a[0] + (b[0]-a[0]) * i / 20)
        y = int(a[1] + (b[1]-a[1]) * i / 20)
        if bw[y, x] == 0:
            return True
    return False

for _ in range(6000):          # Run the RRT* algorithm for a fixed number o
f iterations
    rand = (random.randint(0, 399), random.randint(0, 399))
    # Randomly sample a point in the image

    if not free(rand):
        continue              # Skip if the sampled point is not in free space

    nearest = nodes[0]        # Find the nearest existing node
    for n in nodes:
        if dist(n, rand) < dist(nearest, rand):
            nearest = n

    new = steer(nearest, rand) # Generate a new node towards the rando

```

m sample

```
if not free(new) or collision(nearest, new):  
    continue          # Skip if new node is invalid or path is blocked
```

```
best_parent = nearest  
best_cost = cost[nearest] + dist(nearest, new)  
# Initialize best parent and cost
```

```
for n in nodes:  
    if dist(n, new) < radius and not collision(n, new):  
        c = cost[n] + dist(n, new)  
        if c < best_cost:  
            best_cost = c  
            best_parent = n  
# Choose the best parent within the neighborhood (RRT* optimization)
```

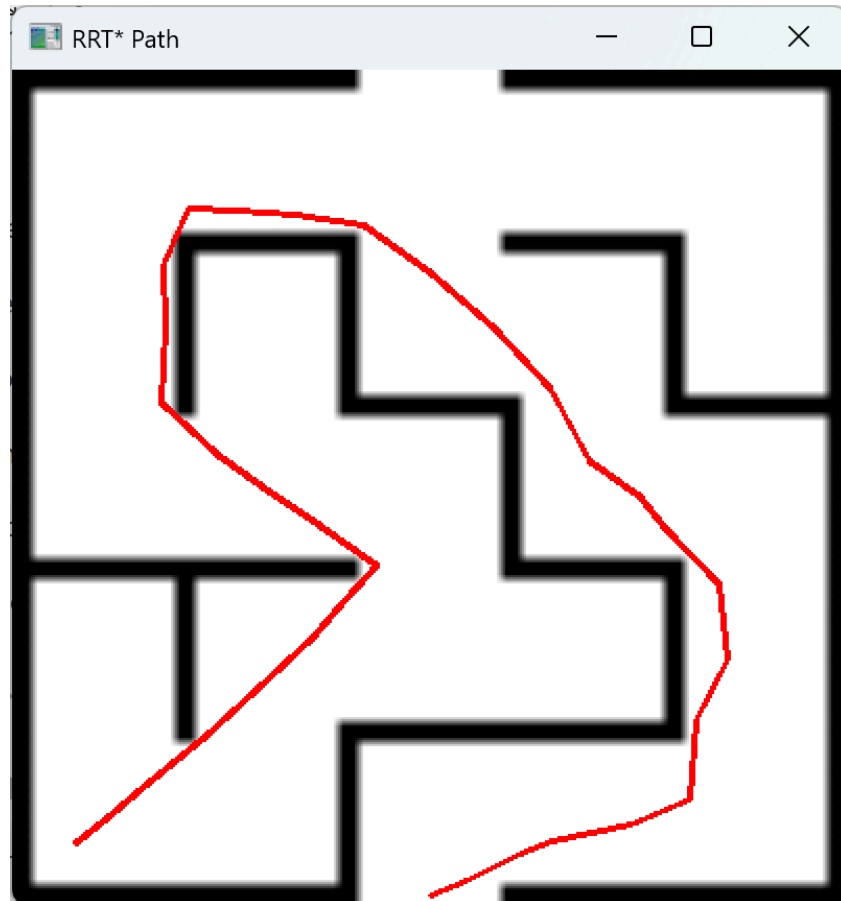
```
nodes.append(new)          # Add the new node to the tree  
parent[new] = best_parent  # Assign its parent  
cost[new] = best_cost      # Store its cost
```

```
for n in nodes:  
    if n != new and dist(n, new) < radius and not collision(new, n):  
        if cost[new] + dist(new, n) < cost[n]:  
            parent[n] = new  
            cost[n] = cost[new] + dist(new, n)  
# Rewire nearby nodes if a cheaper path is found
```

```
if dist(new, goal) < step and not collision(new, goal):  
    parent[goal] = new      # Connect the goal if reachable  
    cost[goal] = cost[new] + dist(new, goal)  
    break
```

```
cur = goal                  # Backtrack from goal to start  
while cur in parent and parent[cur] is not None:  
    p = parent[cur]  
    cv2.line(img, p, cur, (0, 0, 255), 2)  
    # Draw the path segment in red  
    cur = p
```

```
cv2.imshow("RRT* Path", img)      # Display the maze with the RRT* path
cv2.waitKey(0)                    # Wait indefinitely until a key is pressed
cv2.destroyAllWindows()           # Close all OpenCV windows
```



Final Day Project

Python Code

```
import cv2 as cv                # Import OpenCV for computer vision operation
s
import serial                    # Import serial for communication with external h
ardware
import time                     # Import time for delays and FPS calculation
import numpy as np              # Import NumPy for numerical operations
import keyboard as kb           # Import keyboard module for key press det
ection
```

```

import math                # Import math for mathematical computations

coords = (x, y, z)         # Tuple to store coordinates (x, y, z)

l1 = 9.2                   # Length of first arm segment
l2 = 9.8                   # Length of second arm segment

ser = serial.Serial(port='COM8', baudrate=9600, timeout=.1)
# Initialize serial communication on COM8 with baud rate 9600

time.sleep(2)              # Wait for serial connection to stabilize

def Publish(x, y, z):
    try:
        theta0 = math.atan2(y, x)    # Base joint angle
        x = math.hypot(x, y)         # Project x onto horizontal plane
        r = math.hypot(x, z)         # Distance from arm base to target

        D = (r**2 - l1**2 - l2**2) / (2 * l1 * l2)
        # Cosine law for elbow angle

        if D > 1.0:
            D = 1.0
        elif D < -1.0:
            D = -1.0

        D = np.clip(D, -1, 1)        # Clamp D to valid range for acos

        dir = 1
        t = dir * math.sqrt(1 - D**2)
        theta2 = math.atan2(t, D)    # Elbow joint angle

        a = math.atan2(z, x)
        b = math.atan2(l2 * math.sin(theta2), l1 + l2 * math.cos(theta2))

        theta1 = a - b               # Shoulder joint angle

        servo0 = np.clip(s0 + math.degrees(theta0), 0, 180)
        servo1 = np.clip(s1 + math.degrees(theta1), 0, 180)

```

```

servo2 = np.clip(s2 + math.degrees(theta2), 0, 180)
# Convert joint angles to servo angles and clamp to [0, 180]

packet = f"{servo0}, {servo1}, {servo2}\n"
# Format the data packet to send over serial

ser.write(packet.encode('utf-8'))
# Send packet to microcontroller

except KeyboardInterrupt:
    ser.close()          # Close serial connection on interrupt

wCam, hCam = 1200, 980          # Camera resolution settings

cap = cv.VideoCapture(1)        # Open webcam with index 1
cFactor = 15.4 / 1780           # Conversion factor from pixels to real-world
units

cap.set(3, wCam)                # Set camera width
cap.set(4, hCam)                # Set camera height

pTime = 0                      # Initialize previous time for FPS calculation

while True:
    ret, img = cap.read()        # Capture a frame from the camera
    if not ret:
        break

    cTime = time.time()
    fps = 1 / (cTime - pTime + 1e-8) # Compute FPS
    pTime = cTime

    img = cv.flip(img, 1)        # Flip the image horizontally

    cv.putText(
        img,
        f'FPS: {int(fps)}',
        (40, 70),

```

```

    cv.FONT_HERSHEY_COMPLEX,
    3,
    (255, 0, 255),
    3
)
# Display FPS on the image

hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
# Convert the image from BGR to HSV color space

lower = np.array([95, 60, 50]) # Lower HSV bound for color detection
upper = np.array([130, 255, 255]) # Upper HSV bound for color detection

mask = cv.inRange(hsv, lower, upper)
# Create a binary mask for the specified HSV range

kernel = np.ones((5, 5), np.uint8)
mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
# Apply morphological operations to clean up the mask

ys, xs = np.where(mask > 0)
# Get coordinates of detected pixels

if len(xs) > 0 and len(ys) > 0:
    cx_pix = xs.mean() # X centroid in pixel coordinates
    cy_pix = ys.mean() # Y centroid in pixel coordinates

    cx = cx_pix * cFactor * 2 # Convert to real-world X coordinate
    cy = cy_pix * cFactor * 2 # Convert to real-world Y coordinate

    xLen = len(xs) # Number of detected pixels

    k = 1
    c = 0

    if xLen != 0:
        cz = math.sqrt(k * 1080000 / (xLen - 1080000 * c)) * 2
        # Estimate Z coordinate based on object size

```

```

else:
    cz = cy

    print(cx, cy, cz)          # Print computed coordinates
else:
    print("no ball detected")  # No object detected

cv.imshow("Img", img)         # Show original image
cv.imshow("img2", mask)       # Show mask image

if cv.waitKey(20) & 0xFF == 27:
    break                     # Exit loop on ESC key

if kb.is_pressed('ctrl'):
    break                     # Exit loop on CTRL key press

cap.release()                 # Release camera
cv.destroyAllWindows()        # Close all OpenCV windows

```

Arduino Code

```

char receivedData[64];        // Character buffer to store incoming serial data
char* token_abc;              // Pointer used for tokenizing the received string

void setup() {
    Serial.begin(9600);        // Initialize serial communication at 9600 baud rate
}

void loop() {
    if (Serial.available()) {  // Check if data is available on the serial port
        int bytesRead = Serial.readBytesUntil('\n', receivedData, sizeof(receivedData) - 1);
        // Read incoming serial data until newline character or buffer limit is reached

        receivedData[bytesRead] = '\0';
    }
}

```

```

// Null-terminate the received character array to make it a valid C-string

token_abc = strtok(receivedData, ",");
// Extract the first token separated by comma

String token = String(token_abc); // Convert first token to Arduino String
int a = token.toInt();           // Convert first token to integer

int b = (String(strtok(NULL, ","))).toInt();
// Extract second token, convert to String, then to integer

int c = (String(strtok(NULL, ","))).toInt();
// Extract third token, convert to String, then to integer
}
}

```

Python to Serial Monitor Code

```

import serial                #Import serial module for serial communication
import time                  #Import time module for delays

ser = serial.Serial('COM12', 9600, timeout=1)
#Initialize serial communication on COM12 with baud rate 9600 and 1 second
timeout

time.sleep(2)
#Wait for 2 seconds to allow the serial connection (e.g., Arduino) to initialize

try:
    while True:              # Infinite loop to keep sending data
        ser.write(("2,4,8" + '\n').encode('utf-8'))
        #Send the string "2,4,8" followed by a newline over serial

except KeyboardInterrupt:
    ser.close()
    #Close the serial connection when the program is interrupted

```

Declaration

This report is an original work submitted by **Varchas Jasti** as part of the requirements related to the aforementioned event. The content presented reflects personal observations, analysis, and learning outcomes derived from the event. This document is submitted in good faith and is intended solely for academic and evaluative purposes.